

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ**



**BÁO CÁO SƠ BỘ
ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN**

**XÂY DỰNG ỨNG DỤNG QUẢN LÝ CÔNG VIỆC THEO KIẾN
TRÚC ĐA THUÊ BAO**

Mã số đề tài: THS2026-67

Chủ nhiệm đề tài: Sinh viên Ngô Hưng Thịnh

BẢN SƠ BỘ – CHƯA NGHIỆM THU

Cần Thơ, tháng 10 năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ**



BÁO CÁO SƠ BỘ

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN

**XÂY DỰNG ỨNG DỤNG QUẢN LÝ CÔNG VIỆC THEO KIẾN
TRÚC ĐA THUÊ BAO**

Mã số đề tài: THS2026-67

Sinh viên chủ nhiệm đề tài: Ngô Hưng Thịnh

Nam/Nữ: Nam Dân tộc: Kinh

Lớp: DI2396F2

Trường/Khoa/Viện: Trường Công nghệ Thông tin và Truyền thông

Năm thứ: 4 / Số năm đào tạo: 4,5

Ngành học: Kỹ thuật phần mềm (CLC)

Người hướng dẫn: TS. Ngô Bá Hùng

Cần Thơ, tháng 10 năm 2026

Danh sách thành viên nghiên cứu và đơn vị phối hợp chính

STT	Họ và tên	MSSV – Lớp	Vai trò
1	Ngô Hưng Thịnh	B2303904 – DI2396F2	Chủ nhiệm
2	Nguyễn Thanh Trọng	B2305615 – DI2396F1	Thành viên
3	Trang Minh Chánh	B2303864 – DI2396F1	Thành viên
4	Huỳnh Hồng Ân	B2306657 – DI23V7F1	Thành viên
5	Cao Tường Hưng	B2303873 – DI2396F1	Thành viên

Giảng viên hướng dẫn: TS. Ngô Bá Hùng.

Đơn vị phối hợp chính: Không.

Phân công nghiên cứu

Phân công dự kiến theo thuyết minh; đóng góp thực tế cần nhóm xác nhận. Ngô Hưng Thịnh, Nguyễn Thanh Trọng và Cao Tường Hưng tham gia thiết kế, xây dựng ứng dụng và kiến trúc đa thuê bao. Trang Minh Chánh tham gia khảo sát, phân tích yêu cầu và kiểm thử. Huỳnh Hồng Ân tham gia khảo sát, kiểm thử, đánh giá và viết báo cáo. Việc bàn giao từng sản phẩm được đối chiếu với hồ sơ nghiệm thu trước khi công bố.

Mục lục

Danh sách thành viên nghiên cứu và đơn vị phối hợp chính	i
Danh sách bảng	vi
Danh sách hình vẽ	vii
Danh mục từ viết tắt	viii
Thông tin kết quả nghiên cứu của đề tài	ix
Information on research results	xi
Thông tin về sinh viên chủ nhiệm đề tài	xii
Mở đầu	1
Bối cảnh và lý do chọn đề tài	1
Tình hình nghiên cứu và khoảng trống thực hành	1
Mục tiêu và câu hỏi nghiên cứu	2
Đối tượng và phạm vi	2
Phương pháp và tổ chức báo cáo	3
1 Cơ sở lý thuyết và yêu cầu bài toán	4
1.1 Khái niệm và mô hình đa thuê bao	4
1.1.1 Tenant, workspace và người dùng	4
1.1.2 Ba topology dữ liệu và Bridge	4
1.1.3 Cô lập, xác thực và phân quyền	5
1.1.4 Kanban và mô hình nghiệp vụ của nguyên mẫu	5
1.1.5 Nhất quán giao dịch và workflow nền	5
1.2 Tổng quan có cấu trúc và cơ sở lựa chọn nguồn	6
1.2.1 Trạng thái tìm kiếm và sàng lọc	6
1.2.2 Các nghiên cứu nền tảng	6
1.2.3 Từ nguồn đến khoảng trống của dự án	7
1.2.4 Công việc cần hoàn tất cho tổng quan cuối	7
1.3 Yêu cầu, tình huống rủi ro và bất biến	8
1.3.1 Nguồn yêu cầu và vai trò	8
1.3.2 Phạm vi nghiệp vụ cốt lõi	8

1.3.3	Threat model và ranh giới tin cậy	9
1.3.4	Các bất biến dùng cho đánh giá	9
1.3.5	Metric và điều kiện đóng kết luận	9
2	Khung nghiên cứu và phương pháp đánh giá	11
2.1	Khung bài toán và chuỗi truy vết	11
2.1.1	Đơn vị phân tích và câu hỏi thành phần	11
2.1.2	Quy trình sáu bước	11
2.1.3	Phương pháp so sánh và quy tắc lựa chọn	12
2.1.4	Phạm vi phương pháp đã thực hiện	13
2.2	Nguồn dữ liệu và quản lý bằng chứng	13
2.2.1	Bốn lớp bằng chứng	13
2.2.2	Sổ bằng chứng và provenance	13
2.2.3	Đối chiếu source và test	14
2.2.4	Quản lý dữ liệu và thông tin nhạy cảm	14
2.3	Thiết kế kiểm chứng và thực nghiệm	15
2.3.1	Kiểm chứng tính đúng đắn	15
2.3.2	Thiết kế thực nghiệm đề xuất	15
2.3.3	Metric và cách diễn giải	16
2.3.4	Noisy neighbor và phép so sánh cặp	16
2.3.5	Đơn vị lặp, QA và giới hạn thống kê	16
2.3.6	Đánh giá người dùng và điều kiện thực hiện	17
3	Phân tích, lựa chọn và hiện thực kiến trúc	18
3.1	Kiến trúc tổng thể Bridge	18
3.1.1	Tổ chức thành phần và hai plane	18
3.1.2	Mô hình dữ liệu và hợp đồng thống nhất	19
3.1.3	Placement registry và vòng đời schema	19
3.1.4	Quyết định kiến trúc và trạng thái	19
3.2	N1 – Định danh, định tuyến và truyền ngữ cảnh	20
3.2.1	Bài toán và câu hỏi	20
3.2.2	Phương án và đánh đổi	20
3.2.3	Tiêu chí và lựa chọn	21
3.2.4	Hiện thực tại ranh giới request và SQL	21
3.2.5	Bằng chứng, giới hạn và nơi đánh giá	22

3.3	N2 – Bố trí và cô lập dữ liệu	22
3.3.1	Bài toán và hai cấp quyết định	22
3.3.2	So sánh bố trí dữ liệu	22
3.3.3	So sánh enforcement trong Pool	23
3.3.4	Tiêu chí kiểm chứng	23
3.3.5	Hiện thực resolver và runtime role	24
3.3.6	Bảng chứng sàng lọc và diễn giải đúng	24
3.3.7	Giới hạn và công việc còn thiếu	24
3.4	N3 – Xác thực và phân quyền tenant/project	25
3.4.1	Bài toán và câu hỏi	25
3.4.2	Phương án và đánh đổi	25
3.4.3	Tiêu chí và lựa chọn triển khai	25
3.4.4	Hiện thực identity và phiên tenant	26
3.4.5	Hiện thực quyền nội dung	26
3.4.6	Bảng chứng và giới hạn	26
3.5	N4 – Nhất quán trạng thái và xử lý nền	27
3.5.1	Bài toán và câu hỏi	27
3.5.2	Phương án cập nhật đồng thời	27
3.5.3	Phương án chuyển side effect	28
3.5.4	Tiêu chí kiểm chứng	28
3.5.5	Hiện thực mutation và outbox	28
3.5.6	Bảng chứng và giới hạn	29
3.6	N5 – Cấp phát và quản lý tài nguyên dùng chung	29
3.6.1	Bài toán và câu hỏi	29
3.6.2	Phương án cấp phát và lựa chọn	30
3.6.3	Tiêu chí cấp phát và phục hồi	30
3.6.4	Hiện thực state machine và lease	30
3.6.5	Connection pool và ngân sách tài nguyên	31
3.6.6	Storage, quota và cleanup	32
3.6.7	Limiter và quan sát noisy neighbor	32
3.6.8	Bảng chứng, giới hạn và nơi đánh giá	32
4	Bảng chứng kỹ thuật, kế hoạch thực nghiệm và bàn luận	33
4.1	Baseline và hồ sơ kiểm chứng	33
4.1.1	Phiên bản mô tả và môi trường viết	33
4.1.2	Các checkpoint lịch sử được giữ riêng	33

4.1.3	Artifact cũ và kiểm tra mới	34
4.2	Bảng chứng theo nhóm và câu trả lời RQ sơ bộ	34
4.2.1	Phạm vi coverage cần được diễn giải	34
4.2.2	Số test bị bỏ qua và hệ quả	35
4.2.3	Diễn giải sàng lọc isolation	36
4.2.4	RQ1 – Yêu cầu nào cần cho ứng dụng?	36
4.2.5	RQ2 – Xây dựng Bridge dùng chung thành phần thể nào? . . .	37
4.2.6	RQ3 – Kiến trúc đáp ứng đến mức nào?	37
4.2.7	RQ4 – Cơ chế/công nghệ nào phù hợp nhất?	37
4.3	Kế hoạch thu số liệu và bàn luận giới hạn	37
4.3.1	Bảng kết quả cần thu	37
4.3.2	Giới hạn giá trị nội tại	39
4.3.3	Giới hạn giá trị cấu trúc và ngoại tại	39
4.3.4	Nhất quán tài liệu và giới hạn phạm vi	39
4.3.5	Điều kiện hoàn tất bản cuối	40
	Kết luận và kiến nghị	41
	Kết luận trong phạm vi bản sơ bộ	41
	Kiến nghị và công việc tiếp theo	41
	Tài liệu tham khảo	42
A	Tái lập và kiểm toán bằng chứng	44
A.1	Baseline, nguồn và sổ bằng chứng	44
A.2	Môi trường và lệnh tái lập xuất bản	51
A.3	Tái lập kiểm tra kỹ thuật và điều kiện	51
A.4	Manifest cho đợt thực nghiệm sau	51
A.5	Checklist xuất bản và nội dung	52
B	Ma trận quyền, mở rộng và công việc còn thiếu	53
B.1	Ma trận quyền rút gọn	53
B.2	Phản mở rộng ba placement và capability	54
B.3	Các mâu thuẫn đã nhận diện	54
B.4	Điều kiện đóng các khoảng trống	54

Danh sách bảng

1.1	Phân biệt topology với biên được bảo vệ	5
1.2	Vai trò và giới hạn của nguồn trong tổng quan	7
1.3	Nhóm yêu cầu và điều kiện chất lượng	8
2.1	Truy vết nhóm bài toán tới RQ, tiêu chí, hiện thực và đánh giá	12
2.2	Phân loại nguồn và phạm vi mệnh đề được hỗ trợ	13
3.1	So sánh phương án nhận diện tenant	20
3.2	Các cơ chế bảo vệ Pool và đường kiểm chứng	23
3.3	So sánh nguồn quyết định quyền	25
3.4	So sánh cách xử lý cập nhật đồng thời	27
3.5	So sánh phương án cấp phát tenant	30
4.1	Checkpoint kỹ thuật và giới hạn sử dụng	33
4.2	Coverage, bằng chứng và phần chưa đóng	35
4.3	Guard-omission observations lịch sử, không phải benchmark	36
4.4	Trạng thái kết quả performance chưa có dữ liệu	38
4.5	Trạng thái đánh giá bổ sung	38
A.1	Sổ bằng chứng và giới hạn kiểm toán	44
B.1	Phân biệt quyền tenant và quyền project	53
B.2	Một số quyền nội dung project	53
B.3	Capability có thể cấp theo placement local	54
B.4	Điều kiện bổ sung trước kết luận cuối	54

Danh sách hình vẽ

3.1	Kiến trúc control và application plane	18
3.2	Request, tenant context và cleanup	21
3.3	Ba topology dữ liệu trong Bridge	23
3.4	Mutation, outbox và delivery	29
3.5	State machine provisioning	31

Danh mục từ viết tắt

Từ viết tắt	Diễn giải
API	Giao diện lập trình ứng dụng (Application Programming Interface)
SaaS	Phần mềm dưới dạng dịch vụ (Software as a Service)
RLS	Bảo mật ở mức hàng (Row-Level Security)
SLO	Mục tiêu mức dịch vụ (Service Level Objective)



BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ

THÔNG TIN KẾT QUẢ NGHIÊN CỨU CỦA ĐỀ TÀI

BẢN SƠ BỘ – CHƯA NGHIỆM THU

1. Thông tin chung:

- Mã số đề tài: THS2026-67
- Tên đề tài: XÂY DỰNG ỨNG DỤNG QUẢN LÝ CÔNG VIỆC THEO KIẾN TRÚC ĐA THUÊ BAO
- Sinh viên chủ nhiệm đề tài: Ngô Hưng Thịnh
- Lớp: DI2396F2 – Khóa 49; Trường/Khoa/Viện: Trường Công nghệ Thông tin và Truyền thông.
- Năm thứ: 4 Số năm đào tạo: 4,5
- Người hướng dẫn: TS. Ngô Bá Hùng

2. Mục tiêu đề tài:

Thiết kế và hiện thực kiến trúc đa thuê bao cho Kanban; phân tích định danh, cô lập dữ liệu, quyền, nhất quán và quản lý tài nguyên. Xây dựng chuỗi truy vết từ bài toán tới bằng chứng và phương pháp đánh giá.

3. Tính mới và sáng tạo:

Đóng góp sơ bộ là kiến trúc tham chiếu và ma trận kiểm chứng trên cùng nghiệp vụ. Tính mới học thuật và hiệu quả so sánh cần được xác lập qua tổng quan, protocol và dữ liệu; chưa tuyên bố đã xác nhận.

4. Kết quả nghiên cứu:

Code và hồ sơ local mô tả API/worker/web, Bridge ba placement và cơ chế tenant-aware. Bản sơ bộ có phân tích năm nhóm, evidence register và câu trả lời RQ có điều kiện. Chưa có số đo performance, SUS hoặc kết luận placement tối ưu.

5. Sản phẩm:

Nguồn ứng dụng, tài liệu kiến trúc/contract/test và nguồn LaTeX/PDF sơ bộ có trong repository. Bản tin/tóm tắt cùng trạng thái sơ bộ; tình trạng bàn giao chính thức do nhóm xác nhận.

6. Công bố khoa học hoặc nhận xét của cơ sở áp dụng:

Chưa có thông tin công bố hoặc nhận xét cơ sở áp dụng được xác nhận trong hồ sơ.

7. Đóng góp và khả năng áp dụng:

Có thể phục vụ review và học tập kiến trúc đa thuê bao; tác động thực tế, hiệu quả giáo dục và chất lượng dịch vụ chưa được đo.

8. Hiệu quả và phương thức chuyển giao:

Dự kiến bàn giao mã nguồn/tài liệu sau review của nhóm và người hướng dẫn. Vận hành Internet, provider thật và nghiệm thu nghiên cứu còn cần bằng chứng riêng.

Ngày ... tháng ... năm ...

Chủ nhiệm đề tài

Ngô Hưng Thịnh



Nhận xét của người hướng dẫn về những đóng góp khoa học của sinh viên thực hiện đề tài *(phần này do người hướng dẫn ghi):*

.....

Xác nhận của Đại học Cần Thơ

Ngày ... tháng ... năm ...
Người hướng dẫn

TS. Ngô Bá Hùng



INFORMATION ON RESEARCH RESULTS

PRELIMINARY REPORT – NOT A FINAL ACCEPTANCE

1. General information:

Project code: THS2026-67

Project title: Development of a Multi-Tenant Task Management Application

Coordinator: Ngô Hưng Thịnh

Implementing institution: College of Information and Communication Technology, Can Tho University

Reference duration: 6 months, May–October 2026; official dates require confirmation.

2. Objective(s):

Design and implement a multi-tenant Kanban reference architecture. Analyze tenant context, isolation, authorization, consistency and resource management, with traceability from problems to evidence.

3. Contribution:

The preliminary contribution is an implemented reference architecture and a verification framework for one business workflow. Academic novelty and comparative effectiveness require further literature review and registered experiments.

4. Research results:

Source inspection and dated local records describe the API, worker, web client and three data placements. Five problem groups, an evidence register and conditional RQ answers are documented. Performance measurements, SUS and an optimal-placement conclusion remain pending.

5. Products:

Application source, architecture/API/test documents and preliminary LaTeX/PDF outputs are available in the repository. Formal delivery status requires confirmation by the team.

6. Effects, transfer and applicability:

The material supports architecture review and study. Deployment on the public Internet, real providers and measured educational or operational impact require separate evidence; this report does not certify final research acceptance.



BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC CẦN THƠ

THÔNG TIN VỀ SINH VIÊN CHỦ NHIỆM ĐỀ TÀI

I. SƠ LƯỢC VỀ SINH VIÊN:

Họ và tên: Ngô Hưng Thịnh
Sinh ngày: 21 tháng 04 năm 2005
Nơi sinh: Bệnh viện Phụng Hiệp, Cần Thơ
Lớp: DI2396F2 Khóa: 49
Trường/Khoa/Viện: Trường Công nghệ Thông tin và Truyền thông
Địa chỉ liên hệ: 10, đường Lê Hồng Phong, khu vực 3, phường
Ngã Bảy, thành phố Cần Thơ, Việt Nam
Điện thoại: 0939172285
Email: thinhb2303904@student.ctu.edu.vn

Ảnh 4 × 6

II. QUÁ TRÌNH HỌC TẬP

(kê khai thành tích từ năm thứ 1 đến năm đang học):

* Năm thứ 1:

Ngành học: Kỹ thuật phần mềm (CLC) Trường/Khoa/Viện: Trường Công nghệ
Thông tin và Truyền thông.
Kết quả xếp loại học tập:
Sơ lược thành tích:

* Năm thứ 2:

Ngành học: Kỹ thuật phần mềm (CLC) Trường/Khoa/Viện: Trường Công nghệ
Thông tin và Truyền thông.
Kết quả xếp loại học tập:
Sơ lược thành tích:

* Năm thứ 3:

Ngành học: Kỹ thuật phần mềm (CLC) Trường/Khoa/Viện: Trường Công nghệ
Thông tin và Truyền thông.
Kết quả xếp loại học tập:
Sơ lược thành tích:

* Năm thứ 4:

Ngành học: Kỹ thuật phần mềm (CLC) Trường/Khoa/Viện: Trường Công nghệ
Thông tin và Truyền thông.
Kết quả xếp loại học tập:
Sơ lược thành tích:

Xác nhận của Đại học Cần Thơ

Ngày ... tháng ... năm ...
Chủ nhiệm đề tài

Ngô Hưng Thịnh

Mở đầu

Báo cáo sơ bộ, chốt nội dung ngày 09/10/2026. Bản báo cáo mô tả kiến trúc và nguyên mẫu Kanban từ checkout e829f73, tổng hợp hồ sơ kiểm chứng kỹ thuật và thiết kế đợt thực nghiệm tiếp theo. Kết quả local được giới hạn theo ngày, phiên bản và môi trường. Các phép đo hiệu năng, đánh giá người dùng, vận hành Internet và quyết định công nghệ cuối cùng chưa đủ dữ liệu; trạng thái tương ứng là PENDING_DATA. Phân loại này được áp dụng cả trong bản tin và tóm tắt.

Bối cảnh và lý do chọn đề tài

Một dịch vụ quản lý công việc cho nhiều lớp học hoặc nhóm nghiên cứu cần phục vụ đồng thời hai yêu cầu: chia sẻ hạ tầng để vận hành thống nhất và giữ phạm vi dữ liệu, quyền của mỗi không gian làm việc. Một người có thể thuộc nhiều nhóm, nhưng khi đang thao tác ở nhóm A, họ không được đọc dữ liệu nhóm B chỉ vì biết mã dự án hoặc mã task. Biên tenant vì vậy phải được xác lập tại request, truy vấn, lưu trữ và tác vụ nền, thay vì chỉ được biểu diễn bằng một màn hình chọn workspace.

Kanban cung cấp đối tượng nghiệp vụ cụ thể để nghiên cứu ranh giới này: project có thành viên; board có cột; task có trạng thái, người được giao, thời hạn, tài nguyên và lịch sử. Khi hai thành viên cùng sửa task hoặc khi worker gửi nhắc hạn sau một thay đổi, vấn đề không chỉ là quyền đọc mà còn là nhất quán trạng thái và side effect. Ứng dụng được dùng làm nguyên mẫu để kiểm tra kiến trúc trong các tình huống có thể tái lập.

Nghiên cứu này tập trung vào năm nhóm: định danh và truyền ngữ cảnh; bố trí và cô lập dữ liệu; xác thực và phân quyền; nhất quán và xử lý nền; cấp phát và quản lý tài nguyên. Các hướng tiếp cận được đặt trong cùng API và nghiệp vụ để làm rõ đánh đổi. Mục tiêu hiện tại là hoàn thiện lập luận và bằng chứng sơ bộ; chưa khẳng định đã tìm ra placement tối ưu.

Tình hình nghiên cứu và khoảng trống thực hành

Các nguồn đã lựa chọn bao gồm nghiên cứu tổng quan, hướng dẫn phương pháp và tài liệu kỹ thuật chính thức. Olabanji và cộng sự lập bản đồ nghiên cứu multi-tenancy trong cloud-native; Narasayya và Chaudhuri khảo sát dịch vụ dữ liệu đám mây, trong đó cô lập hiệu năng và tài nguyên là các vấn đề cần đánh giá [1], [2]. Phạm vi của hai công trình rộng hơn nguyên mẫu Kanban này.

Khoảng trống được phát biểu ở mức dự án: cần một chuỗi truy vết từ yêu cầu đến

lựa chọn kiến trúc, mã thực thi, tình huống kiểm tra và điều kiện đo trên tài nguyên giới hạn. Báo cáo không tuyên bố chưa từng có hệ thống tương tự trong toàn bộ học thuật. Tổng quan hiện là tổng hợp có cấu trúc; nhật ký tìm kiếm của nhóm chưa có đầy đủ export, sàng lọc hai người và extraction cho một systematic mapping hoàn chỉnh.

Mục tiêu và câu hỏi nghiên cứu

Mục tiêu tổng quát là thiết kế, hiện thực và đánh giá một kiến trúc tham chiếu đa thuê bao cho quản lý công việc. Mục tiêu cụ thể gồm xác lập ngữ cảnh tenant có kiểm chứng, bảo vệ dữ liệu và quyền, phát hiện xung đột cập nhật, phục hồi workflow cấp phát và chuẩn bị phép đo có truy vết.

Bốn câu hỏi trong protocol 0.1 được giữ nguyên về ý nghĩa và phạm vi baseline E02:

1. **RQ1:** Ứng dụng quản lý công việc trong môi trường đại học cần các yêu cầu nghiệp vụ và đa thuê bao nào?
2. **RQ2:** Làm thế nào xây dựng khung Bridge kết hợp Pool và Silo nhưng vẫn dùng chung danh tính, cấp phát, mã nghiệp vụ và trải nghiệm vận hành?
3. **RQ3:** Kiến trúc đáp ứng đến mức nào các yêu cầu về cô lập dữ liệu, phân quyền, cấp phát, hiệu năng, kiểm soát noisy neighbor và khả dụng?
4. **RQ4:** Cơ chế và công nghệ nào phù hợp nhất để hiện thực kiến trúc trên một VPS tài nguyên giới hạn?

N1–N5 là các câu hỏi thành phần giúp tổ chức lập luận, không thay thế RQ đã đăng ký. Việc thêm Schema-per-tenant vào hiện thực được mô tả là phần mở rộng local; phép đo ba placement phải có phụ lục protocol đăng ký trước.

Đối tượng và phạm vi

Đối tượng nghiên cứu là các ranh giới tenant trong luồng Kanban, từ phiên đăng nhập đến SQL và worker. Hệ thống hiện có API/worker Spring Boot, frontend React, PostgreSQL và các adapter local. Application plane hỗ trợ Pool, Schema-per-tenant và Silo database; control plane dùng chung tài khoản và metadata vận hành.

Phạm vi bản sơ bộ bao gồm phân tích mã, hợp đồng OpenAPI, ADR, test và biên bản local. Capability và tùy biến được nhận diện trong phụ lục với trạng thái học thuật

chưa xác nhận. Chưa đưa realtime collaboration, autoscaling, full-stack silo, di chuyển tenant có dữ liệu hoặc chứng nhận production vào kết luận.

Thuyết minh mục 5 ghi tháng 5–10/2026, còn mục 13.2 và tiến độ ghi tháng 3–8/2026. Biểu mẫu hiện giữ khoảng thời gian ở mục 5 làm thông tin tham chiếu; thời hạn chính thức vẫn cần đối chiếu hồ sơ đã ký. Tương tự, IEEE được dùng cho bản sơ bộ trong khi quy cách tài liệu tham khảo cuối cần nhóm xác nhận E01.

Phương pháp và tổ chức báo cáo

Phương pháp gồm phân tích tài liệu có kiểm chứng, xác lập tình huống và bất biến, so sánh phương án, đối chiếu hiện thực, tổng hợp bằng chứng và thiết kế đánh giá. Một test dùng fixture cung cấp bằng chứng cho hành vi trong tình huống đó; không cung cấp số đo chất lượng dịch vụ trên VPS hoặc đánh giá người dùng.

Chương 1 trình bày khái niệm, tổng quan và yêu cầu. Chương 2 xây dựng chuỗi truy vết và phương pháp đánh giá. Chương 3 phân tích năm nhóm và kiến trúc hiện thực. Chương 4 tổng hợp bằng chứng, trả lời RQ sơ bộ và mô tả phép đo còn thiếu. Phụ lục lưu đường dẫn kiểm toán, ma trận quyền và giới hạn của phần mở rộng.

Chương 1

Cơ sở lý thuyết và yêu cầu bài toán

Chương này phân biệt đặc điểm kiến trúc, cơ chế hiện thực và yêu cầu chất lượng. Từ đó, các tình huống nghiệp vụ được chuyển thành bất biến để so sánh giải pháp và kiểm tra ở các chương sau.

1.1 Khái niệm và mô hình đa thuê bao

1.1.1 Tenant, workspace và người dùng

Trong mô hình của đề tài, tenant là đơn vị có phạm vi dữ liệu, membership và vận hành riêng; workspace là cách gọi đơn vị ấy ở giao diện. Account là danh tính chung của một người. Quan hệ account–tenant là nhiều–nhiều, còn quyền project được gán ở một phạm vi nhỏ hơn. Một account có vai trò Owner tại A và Member tại B không được mang quyền của A sang phiên B.

Đặc điểm đa thuê bao nằm ở việc nhiều workspace sử dụng cùng hệ thống. Cơ chế như host/token, RLS hay quota là cách hiện thực đặc điểm đó. Yêu cầu chất lượng là điều kiện cần kiểm chứng, chẳng hạn từ chối truy cập chéo hoặc bảo đảm một lần retry không tạo thêm tài nguyên. Trộn ba tầng này sẽ dẫn tới kết luận thiếu căn cứ: việc có RLS trong migration chưa chứng minh mọi đường dữ liệu đều cô lập.

1.1.2 Ba topology dữ liệu và Bridge

Thuật ngữ placement chỉ vị trí tổ chức dữ liệu nghiệp vụ, không đồng nghĩa một máy chủ vật lý. Pool dùng chung database, schema và bảng. Schema-per-tenant dùng database chung nhưng schema riêng. Silo database cấp database riêng, trong khi API, worker, storage và host PostgreSQL vẫn có thể dùng chung. Bridge là hệ thống kết hợp các placement sau một hợp đồng ứng dụng. Mô hình compute chung/database riêng cũng được tài liệu targeted isolation của AWS phân tích [3].

Việc nhiều đơn vị migration và pool xuất hiện ở Schema/Silo là hệ quả cấu trúc có thể suy ra từ hiện thực. Mức chi phí thực tế, độ trễ hoặc khả năng chịu tải không thể suy ra chỉ từ bảng này; cần phép đo cùng điều kiện.

Bảng 1.1: Phân biệt topology với biên được bảo vệ

Placement	Cấu trúc application data	Biên và phần vẫn chia sẻ
Pool	Chung database/schema/bảng; mỗi hàng có tenant ID	Cần enforcement theo hàng; cùng database và tài nguyên máy chủ
Schema-per-tenant	Schema và runtime role riêng trong database chung	Namespace cộng quyền schema; vẫn chung database/CPU/I/O
Silo database	Database và runtime role riêng	Biên database; vẫn có thể chung API, worker, storage và host

1.1.3 Cô lập, xác thực và phân quyền

Xác thực trả lời ai đang gửi request. Định tuyến tenant xác định workspace và placement. Phân quyền quyết định hành động nào được phép với tài nguyên cụ thể. Nhận diện tenant và quyền người dùng là trách nhiệm riêng; định danh lấy từ request phải được kiểm tra với thông tin phiên trước khi cấp quyền [4].

Trong hiện thực, tenant context kết hợp user ID, tenant ID, slug, tier, placement và mã truy vết. Các guard ứng dụng hạn chế hành động và tập dữ liệu; chính sách database bổ sung enforcement ở đường SQL. Storage guard bảo vệ object key. Worker phải tái lập context từ registry cho từng tenant vì vòng đời job khác vòng đời request E10–E14.

1.1.4 Kanban và mô hình nghiệp vụ của nguyên mẫu

Board của nguyên mẫu thể hiện công việc bằng các cột và task; thao tác di chuyển thay đổi trạng thái. Hệ thống hỗ trợ nhiều board trong project, task có assignee/deadline/priority, subtask một cấp, bình luận và file/link resource. Đây là phạm vi nghiệp vụ được mô tả trong SRS và API, không phải phép đánh giá năng suất làm việc của người dùng E01, E15.

Các quan hệ có tác dụng kiểm soát biên: board thuộc project, task thuộc board/project, project member có role, resource liên kết nhiều task cùng tenant. Khi nhận task ID, server cần kiểm tra quan hệ sở hữu và membership; kiểm tra hình dạng UUID chỉ bảo đảm cú pháp. Một resource được chia sẻ qua nhiều task cũng không làm mất yêu cầu authorize người tải tại từng project liên quan.

1.1.5 Nhất quán giao dịch và workflow nền

Một mutation task và việc tạo event có thể nằm trong cùng application transaction. Gửi SMTP hoặc tạo database là side effect bên ngoài transaction ấy. Thiết kế outbox nhằm

giải quyết rủi ro hai thao tác ghi dữ liệu/phát sự kiện tách rời; consumer vẫn cần xử lý delivery lặp [5].

Nguyên mẫu dùng optimistic version để phát hiện client cập nhật từ phiên bản cũ, dùng outbox để chuyển công việc sang worker và dùng state machine để quản lý provisioning. Các cơ chế giải quyết những vấn đề khác nhau: version không thay thế authorization; outbox không chứng minh exactly-once end-to-end; state machine không tự bảo đảm rollback của side effect đã thành công. Tiêu chí của từng cơ chế được tách trong N3–N5.

1.2 Tổng quan có cấu trúc và cơ sở lựa chọn nguồn

1.2.1 Trạng thái tìm kiếm và sàng lọc

Kho nguồn hiện có gồm source register, BibTeX và pilot discovery. Chưa có bộ export đầy đủ từ ACM, IEEE, SpringerLink, ScienceDirect và Google Scholar, chưa hoàn tất sàng lọc hai người hoặc số lượng flow. Vì vậy báo cáo này là tổng quan có cấu trúc, chưa phải systematic mapping đã hoàn thành. Hướng dẫn Petersen được dùng để tổ chức các bước tìm kiếm, lựa chọn, phân tích và trình bày; PRISMA chỉ hỗ trợ minh bạch báo cáo [6], [7].

Các nguồn được đọc theo ba lớp. Nguồn học thuật cung cấp khung khái niệm và phương pháp; tài liệu chính thức mô tả cơ chế kỹ thuật; tài liệu sản phẩm hỗ trợ xác định yêu cầu nhưng không cho phép suy ra kiến trúc database không được công bố. Metadata hợp lệ chỉ là điều kiện nhận diện nguồn, không phải đánh giá chất lượng hoặc bằng chứng về hiệu quả của nguyên mẫu.

1.2.2 Các nghiên cứu nền tảng

Olabanji, Fitch và Matthew báo cáo 921 bản ghi ban đầu, chọn 64 công bố giai đoạn 2015–2022 và phân loại nghiên cứu multi-tenancy trong cloud-native [1]. Kết quả ấy thuộc nghiên cứu của tác giả; không được ghi thành số lượng tìm kiếm của nhóm. Nguồn hữu ích để phát triển từ khóa và chiều phân loại, nhưng không trả lời trực tiếp cách tổ chức quyền project trong Kanban.

Narasayya và Chaudhuri khảo sát kiến trúc cloud data services, elasticity, SLA, performance isolation và chi phí [2]. Báo cáo dùng nguồn này để xác định các chiều cần đo; chưa vận dụng kết quả của dịch vụ database đám mây làm số đo cho VPS của đề tài.

Pushpan có metadata được đối chiếu, DOI 10.32628/CSEIT241061151, trang 1117–1126 [8]. Thuyết minh đang ghi trang khác. Trong bản sơ bộ, nguồn này chỉ hỗ

trợ nhận diện chủ đề; chất lượng bằng chứng chưa được chấm và không tham gia quyết định chọn công nghệ cuối cùng. Nguồn Kumar thiếu venue/DOI/bản gốc và hai tài liệu trong nước thiếu metadata chính thức được giữ ngoài lập luận chính.

Bảng 1.2: Vai trò và giới hạn của nguồn trong tổng quan

Nguồn	Vai trò trong đề tài	Giới hạn suy luận
Olabanji và cộng sự	Điểm xuất phát cho taxonomy và từ khóa	Cloud-native rộng hơn SaaS nghiệp vụ; không thay sàng lọc của nhóm
Narasayya–Chaudhuri	Chiều đánh giá isolation, SLA, tài nguyên	Không có số đo của nguyên mẫu Kanban
Petersen; PRISMA	Tổ chức phương pháp và báo cáo tìm kiếm	Chưa có export/flow để xác nhận mapping hoàn tất
Tài liệu PostgreSQL	Điều kiện hoạt động của RLS và schema	Cần đối chiếu runtime role và transaction thực tế
Microsoft/AWS	Phương án routing, identity, outbox, chia sẻ tài nguyên	Hướng dẫn kiến trúc, không phải benchmark của hệ thống này

1.2.3 Từ nguồn đến khoảng trống của dự án

Tổng quan tạo ra các chiều phân tích: tenant context xuyên tầng; enforcement theo hàng/namespace/database; quyền có phạm vi; mutation và side effect; giới hạn tài nguyên chia sẻ. Khoảng trống thực hành là chưa có đủ bằng chứng đo trong checkout để đánh giá các chiều này đồng thời trên cùng nghiệp vụ và cấu hình giới hạn. Chương 3 đặt các phương án vào năm nhóm, thay vì liệt kê công nghệ không gắn tình huống.

Đóng góp sơ bộ có thể đánh giá là một kiến trúc hiện thực cùng chuỗi truy vết và tình huống kiểm chứng. Đóng góp về placement tốt hơn, chi phí thấp hơn hoặc tác động học tập tốt hơn vẫn chờ dữ liệu. Việc có ba placement trên local là khả năng kỹ thuật đã hiện thực, còn so sánh học thuật ba placement cần được đăng ký bổ sung trước khi thu số liệu.

1.2.4 Công việc cần hoàn tất cho tổng quan cuối

Nhóm cần khóa ngày tìm kiếm, lưu chuỗi truy vấn theo từng cơ sở dữ liệu, khử trùng theo DOI rồi title/year, sàng lọc title/abstract và full text, lưu lý do loại và giải quyết bất đồng. Extraction cần ghi câu hỏi, phương pháp, môi trường, kết quả, hạn chế và quan hệ với RQ. Số lượng flow được tính từ register và export; không lấy số của bài tổng quan khác làm số của nhóm.

1.3 Yêu cầu, tình huống rủi ro và bất biến

1.3.1 Nguồn yêu cầu và vai trò

Yêu cầu được truy về thuyết minh, SRS, permission matrix và OpenAPI E01, E15. RQ1 hỏi yêu cầu nào cần cho môi trường đại học; hồ sơ hiện hỗ trợ phân tích yêu cầu, chưa hỗ trợ kết luận nhu cầu của một mẫu người dùng đã khảo sát. Các phát biểu về mức hài lòng hoặc hiệu quả giáo dục vì vậy không được rút từ chức năng đã cài đặt.

Vai trò hệ thống quản trị metadata vận hành; Owner/Admin tenant quản lý workspace và membership; Manager/Member/Viewer project quản lý hoặc tương tác với nội dung theo quyền. Owner không tự có quyền nội dung mọi project. Một tài khoản có nhiều role được xét trong tenant và project hiện hành, không hợp nhất quyền giữa các tenant.

1.3.2 Phạm vi nghiệp vụ cốt lõi

Onboarding gồm đăng ký/đăng nhập, chọn tenant, payment local, provisioning và trao đổi phiên vào host tenant. Luồng nghiệp vụ gồm lời mời, project membership, project lifecycle, board/column, task và bình luận. Resource có file và link, quota, liên kết task và xóa mềm. Notification gồm in-app, preference, subscription và delivery qua adapter. Mã nguồn có mở rộng hữu hạn, nhưng phần này được trình bày riêng để giữ ranh giới phạm vi.

Bảng 1.3: Nhóm yêu cầu và điều kiện chất lượng

Nhóm	Yêu cầu truy vết	Điều kiện quan sát
N1	FR-01–06; SEC-01–09	Host/token/membership/status cùng tenant; context được dọn sau request
N2	ARC-04; SEC-10–12	Không đọc/ghi/xóa chéo ở case kiểm tra; dữ liệu tenant khác giữ nguyên
N3	FR-20–45; permission matrix	Allow/deny theo hành động; revoke có hiệu lực; deny không tạo side effect
N4	FR-42/45/60–63; DATA-05	Conflict phiên bản cũ; batch nguyên tử; event lặp/stale được xử lý đúng
N5	FR-50–55/70–78; REL-01–03	Provisioning phục hồi; rollback đúng ownership; tài nguyên có giới hạn và quan sát

1.3.3 Threat model và ranh giới tin cậy

Mô hình xem client là nguồn input cần kiểm tra: Host, ID tài nguyên, expected version, object key và payload có thể bị sửa. Người dùng có thể biết ID của tenant khác hoặc giữ access token sau khi bị revoke. Request và job có thể thất bại giữa các bước; connection và thread có thể được tái sử dụng. Các rủi ro này được kiểm tra bằng negative case, không chỉ bằng luồng UI thuận lợi.

Các ranh giới cần bảo vệ gồm trình duyệt-API, API-control database, executor-application placement, ứng dụng-object storage và worker-dịch vụ ngoài transaction. Credential đặc quyền của provisioner được tách khỏi API. Hệ thống nguyên mẫu chưa có bằng chứng nghiệm thu cấu hình proxy/DNS/TLS công khai; mô hình ở đây không tuyên bố kiểm chứng mọi tấn công hạ tầng hoặc host administrator độc hại.

1.3.4 Các bất biến dùng cho đánh giá

1. **I1 – Ngăn cản:** business SQL chỉ được gọi khi context đã xác thực; host và token của hai tenant khác nhau phải bị chặn trước nghiệp vụ.
2. **I2 – Biên dữ liệu:** hành động của tenant A không trả nội dung hoặc thay đổi hàng/object thuộc B trong case được kiểm tra.
3. **I3 – Quyền:** role hợp lệ cùng membership hiện hành quyết định hành động; client không tự nâng quyền bằng payload hoặc UI.
4. **I4 – Cập nhật:** expected version cũ tạo conflict; một batch thất bại không để lại thay đổi một phần.
5. **I5 – Side effect:** retry/callback/event lặp không nhân bản kết quả nghiệp vụ đã được deduplicate; event stale không gửi cho assignee cũ.
6. **I6 – Cấp phát:** tenant chỉ ACTIVE sau khi placement/migration/route đủ điều kiện; compensation không xóa tài nguyên tenant khác.

Từng bất biến được gắn với code và evidence ID ở Bảng 2.1. Điều kiện zero-cross-tenant ở đây là tiêu chí loại cho tập tình huống, không phải ước lượng xác suất bằng không cho toàn bộ hệ thống.

1.3.5 Metric và điều kiện đóng kết luận

Tính đúng đắn được quan sát bằng mã phản hồi, trạng thái trước/sau, số event/job/object và membership hiệu lực. Hiệu năng cần p50/p95, throughput, lỗi, HTTP 429, CPU, RAM

và connections theo tenant. Khả dụng và phục hồi cần fault model, thời điểm lỗi, thời gian khôi phục và trạng thái dữ liệu. Không gán SLO trước pilot trên môi trường khóa.

Bản sơ bộ hoàn thành khi đủ lập luận và phần thiếu có trạng thái cùng điều kiện bổ sung. Kết luận cuối còn phụ thuộc protocol, số đo, review ADR và phạm vi được xác nhận. Một báo cáo biên dịch đạt không thay cho các công nghiên cứu. Yêu cầu tối thiểu 50 trang nội dung của bản tổng kết được theo dõi riêng; không dùng trang trống hoặc kết quả giả để đạt số trang.

Chương 2

Khung nghiên cứu và phương pháp đánh giá

Khung nghiên cứu chuyển yêu cầu thành câu hỏi thành phần, so sánh phương án và xác định loại bằng chứng có thể hỗ trợ kết luận. Phương pháp đã áp dụng cho bản sơ bộ được phân biệt với các đợt đo mới chỉ được thiết kế.

2.1 Khung bài toán và chuỗi truy vết

2.1.1 Đơn vị phân tích và câu hỏi thành phần

Đơn vị phân tích kiến trúc là một đường xử lý có tenant: request, transaction, object operation hoặc job. Đơn vị quan sát kỹ thuật là tình huống kiểm tra cùng trạng thái trước/sau. Đơn vị thực nghiệm hiệu năng dự kiến là một run độc lập trên cấu hình đã khóa; request trong cùng run không được mặc nhiên coi là các lần lặp độc lập.

N1 hỏi cách giữ một ngữ cảnh đúng xuyên vòng đời xử lý. N2 hỏi cách bố trí và bảo vệ dữ liệu trên hạ tầng giới hạn. N3 hỏi cách thực thi quyền theo tenant/project và khi membership thay đổi. N4 hỏi cách phát hiện cập nhật cũ và xử lý side effect lặp/stale. N5 hỏi cách cấp phát có thể phục hồi và chia sẻ tài nguyên có kiểm soát. Các câu hỏi được lấy từ tình huống thất bại cụ thể, không lấy tên công nghệ làm câu hỏi nghiên cứu.

2.1.2 Quy trình sáu bước

1. **Xác định bài toán:** mô tả tác nhân, tài nguyên, phạm vi tin cậy và tình huống gây sai lệch.
2. **Phương án:** liệt kê ít nhất hai cách giải quyết cùng vấn đề; tách topology dữ liệu khỏi cơ chế enforcement.
3. **Tiêu chí:** ghi điều kiện allow/deny, side effect, phục hồi hoặc metric cần đo; không đặt ngưỡng sau khi biết kết quả.
4. **Hiện thực:** đối chiếu component, SQL, migration và vòng đời transaction của checkout; phân biệt thiết kế dự kiến với code đang chạy.
5. **Bằng chứng:** liên kết source/test/biên bản với ngày, phiên bản, môi trường, kết quả và giới hạn.

6. **Mục báo cáo:** gắn phân tích vào Chương 3, kết quả/giới hạn vào Chương 4 và đường kiểm toán vào phụ lục.

Lý do lựa chọn được viết giữa bước tiêu chí và hiện thực. “Đã chọn để cài đặt” là một quyết định phát triển; “phù hợp nhất” là một kết luận so sánh đòi hỏi bằng chứng bổ sung. Một ADR còn Proposed không được chuyển thành Accepted chỉ vì nội dung báo cáo đã hoàn thiện.

Bảng 2.1: Truy vết nhóm bài toán tới RQ, tiêu chí, hiện thực và đánh giá

Nhóm	RQ / mục	Phương án và tiêu chí trọng tâm	Hiện thực / bằng chứng
N1	RQ2/3/4; 3.2	Host/path/header/token; mismatch trước SQL, dọn context khi lỗi	Filter/holder/executor; E10; test hiện có, giới hạn ở 4.2
N2	RQ2/3/4; 3.3	Ba topology; predicate/ORM/RLS; đọc/ghi/xóa chéo và quyền runtime	Resolver/migrations; E09, E11; chưa có phép đo placement
N3	RQ1/2/3; 3.4	JWT role/server membership/ABAC; allow/deny, revoke, không side effect	Auth/project guard; E12, E05; phụ lục quyền
N4	RQ2/3; 3.5	LWW/version/locking; direct send/outbox/broker; conflict, batch, dedupe, stale	Project service/outbox/reminder; E13, E06
N5	RQ2/3/4; 3.6	Manual/synchronous/worker; retry, lease, ownership, quota/pool/limiter	Provisioner/storage/resolver; E08, E14; phép đo ở 4.3

2.1.3 Phương pháp so sánh và quy tắc lựa chọn

Phân tích định tính xem xét correctness, khả năng kiểm tra, vận hành và độ phức tạp. Điều kiện bảo mật là điều kiện loại: một truy cập chéo thành công không được bù bằng điểm hiệu năng tốt. Các trọng số isolation/storage đã đăng ký trong ADR/spike plan được giữ cho đợt nghiên cứu tương ứng; bản sơ bộ không tạo scorecard mới từ nhận xét chủ quan.

So sánh placement yêu cầu cùng phiên bản nghiệp vụ, dữ liệu tương đương và workload. So sánh enforcement Pool là một phép so sánh khác, có deliberate guard omission và kiểm tra runtime role. Không gọi “RLS so với Silo” là hai phương án cùng loại: RLS là cơ chế bảo vệ, Silo là topology. Chính sự phân biệt này giúp diễn giải sáu leak trong harness mà không suy rộng sang mọi đường nghiệp vụ.

2.1.4 Phạm vi phương pháp đã thực hiện

Bản sơ bộ đã sử dụng đọc tài liệu, kiểm tra source, đối chiếu tên test và tổng hợp checkpoint. Kiểm tra công cụ xuất bản được thực hiện trên môi trường hiện tại. Tình trạng lượt backend mới được ghi riêng trong E04; frontend và E2E không được mặc nhiên xem là chạy lại. Workload, fault injection, noisy-neighbor, pilot và thu dữ liệu người dùng vẫn chưa được mở lại.

2.2 Nguồn dữ liệu và quản lý bằng chứng

2.2.1 Bốn lớp bằng chứng

Bảng 2.2: Phân loại nguồn và phạm vi mệnh đề được hỗ trợ

Loại	Có thể hỗ trợ	Không tự hỗ trợ
Source/contract	Component, cấu trúc dữ liệu, thuật toán, đường gọi	Hành vi runtime hoặc hiệu năng đã đạt
Biên bản lịch sử	Kết quả được ghi tại ngày/phiên bản/môi trường cũ	Kiểm chứng lại checkout hiện tại
Artifact hiện có	Số test, lỗi, skip và thuộc tính được lưu trong artifact	Commit thực thi nếu chưa có provenance
Kiểm tra mới	Kết quả của lệnh thực sự chạy, cùng môi trường hiện tại	Cổng nghiên cứu, benchmark hoặc production ngoài phạm vi lệnh

Dữ liệu thực nghiệm chính thức là một lớp riêng còn thiếu trong checkout. Các loại fixture, development và pilot không được đổi nhãn thành experiment. Mệnh đề kết luận theo protocol được gắn MEASURED, INFERRED, LIMITATION hoặc PENDING_DATA; riêng số test là kết quả kiểm tra kỹ thuật, không phải MEASURED performance.

2.2.2 Sổ bằng chứng và provenance

Mỗi bản ghi gồm evidence ID, mệnh đề, nguồn, ngày, commit nếu xác lập được, môi trường, kết quả, skipped count và giới hạn. Sổ chi tiết ở docs/research/preliminar

y-report-evidence.md; bản đọc trong PDF ở Phụ lục A. Mã evidence là đường truy vết, không phải nhãn tự chứng nhận mức tin cậy.

Checkout viết báo cáo là e829f73 trên main. Working tree đã có thay đổi report trước lượt này; baseline mã ứng dụng và nội dung LaTeX chưa commit được ghi riêng. Metadata XML cũ có mốc sửa tệp ngày 24/09/2026 theo Asia/Taipei, nhưng mtime không đủ xác nhận ngày chạy hoặc commit. Vì vậy E03 giữ provenance chưa xác lập; không ghép artifact với checkout chỉ vì tổng test trùng biên bản.

Biên bản EXT 08/09/2026 ghi application V8, 96 backend test, 16 frontend test và 3 E2E. Biên bản nhắc hạn có tên file ngày 18/09 nhưng phần đầu ghi ngày 19/09/2026, V12, 106 backend test và 22 frontend test. Bản sơ bộ giữ hai checkpoint riêng và nêu chênh lệch tên file/ngày nội dung E05, E06.

2.2.3 Đối chiếu source và test

Đọc source xác định điểm thực thi tenant binding, transaction local settings, role checks, version predicate, enqueue outbox và claim provisioning. Đọc test xác định assertion thật: đọc/ghi/xóa chéo, trạng thái tenant B giữ nguyên, rollback sau batch fail, revoke token cũ, assignee stale và ownership của tài nguyên. Sự tồn tại của tên test chỉ cho biết có tình huống được thiết kế; kết quả phải lấy từ report của lần chạy tương ứng.

Phạm vi được kiểm tra cần ghi cả đường truy cập. Một assertion ORM không bao phủ native SQL; một request case không bao phủ job nền; một role allow không bao phủ deny và revoke; một Pool/Silo case không tự bao phủ Schema-per-tenant. Các khoảng trống được đưa vào bảng coverage thay vì suy ra từ tổng số test.

2.2.4 Quản lý dữ liệu và thông tin nhạy cảm

Repo nhận phương pháp, manifest schema, liên kết artifact và kết quả tổng hợp phù hợp. Secrets, raw measurements và dữ liệu người tham gia được lưu ngoài Git theo yêu cầu công việc này. Không đưa token, credential, signed URL đang hiệu lực hoặc dữ liệu nội dung thật vào ví dụ. Các sơ đồ được dựng từ source và tài liệu, mang ý nghĩa cấu trúc, không mang ý nghĩa số đo.

Khi có run mới, cần lưu input bất biến, checksum và metadata môi trường; công cụ phân tích tạo derived output mà không sửa raw input. Run thiếu tệp, sai protocol, thay seed hoặc không đủ metric được gán cờ và giải thích. Không tự xóa outlier để làm đẹp kết quả; quy tắc loại cần đăng ký trước.

2.3 Thiết kế kiểm chứng và thực nghiệm

2.3.1 Kiểm chứng tính đúng đắn

Mỗi tiêu chí có positive và negative case. Với N1, gửi token A đến host B, giữ token sau revoke, suspend tenant và phát sinh exception để kiểm tra cleanup. Với N2, dùng ID của B để đọc/ghi/xóa ở A, bỏ guard ở native/bulk/background và kiểm tra connection sau commit/rollback. Với Schema-per-tenant, còn cần SQL ghi rõ schema B; search path đúng không đủ thay thế quyền runtime.

N3 kiểm tra ma trận role theo hành động bằng API trực tiếp, quyền Owner không có project membership và deny không tạo event/row. N4 kiểm tra hai client cùng expected version, batch có một item stale, delivery lặp và assignee/deadline đổi trước dispatch. N5 kiểm tra callback trùng, claim đồng thời, worker crash sau DDL/migration, lease recovery, rollback lỗi và tài nguyên tenant khác không bị dọn.

Các tình huống fault injection nêu trên là nội dung hồ sơ hoặc thiết kế đánh giá; trong lượt viết báo cáo không tự chạy lại harness đang tạm dừng. Kiểm chứng tích hợp hiện tại chỉ được coi là đóng khi suite yêu cầu Docker thực thi đầy đủ và report có truy vết phiên bản.

2.3.2 Thiết kế thực nghiệm đề xuất

Thiết kế dự kiến sử dụng một VPS có cấu hình cố định, hai mức 3 và 5 tenant, mỗi mức 10 và 20 VU/tenant, ít nhất ba run độc lập mỗi tổ hợp. Mỗi run tách warm-up 2 phút và cửa sổ đo 5 phút. Ba placement được chạy riêng với dữ liệu tương đương, xen kẽ thứ tự chạy; capability tùy biến tắt khi so sánh nghiệp vụ lỗi. Đây là đề xuất chờ pilot/phụ lục protocol, chưa phải tham số đã được duyệt hoặc SLO.

Nếu xét đủ ba placement, hai mức tenant, hai mức VU và ba lần lặp, thiết kế có 36 run đo. Số lượng này chỉ là hệ quả của thiết kế factorial đề xuất, không phải số run đã thực hiện. Việc chọn quy mô cần kiểm tra lại với tài nguyên VPS và độ biến thiên của pilot.

Workload đọc hiện có là `GET/api/v1/projects?size=20`. Trong `experiments/k6/baseline.js`, mỗi tenant có scenario constant-vus và sleep mặc định một giây. Đây là điểm xuất phát phù hợp với câu hỏi số VU đồng thời. `experiments/k6/load.js` dùng constant-arrival-rate với trần VU khác, do đó đo một mô hình tải khác; không dùng maxVUs 100 để gọi đó là thí nghiệm 10–20 người dùng đồng thời.

2.3.3 Metric và cách diễn giải

Độ trễ p50/p95 phải được báo cáo per-tenant và per-run, cùng throughput, số request và error rate. HTTP 429 được tách khỏi lỗi hệ thống vì request bị limiter từ chối là một hành vi cần quan sát. Throughput được định nghĩa theo request hoàn thành trong cửa sổ đo; successful throughput cần ghi riêng khi có nhiều request thất bại. Không cộng hoặc lấy trung bình các p95 rồi gọi đó là p95 gộp.

Tài nguyên cần CPU, RSS/heap nếu có, RAM của container, connections đang mở hoặc active, pool waiting và trạng thái job nền. Manifest ghi pool size của control plane, API và worker, cùng placement, seed, phiên bản JVM, PostgreSQL, k6, image digest và các dịch vụ cùng chạy. Endpoint list projects còn phụ thuộc số hàng và membership; hai seed có kích thước khác nhau không tạo phép so sánh tương đương.

Workload đọc chỉ hỗ trợ kết luận cho endpoint đọc. Mutation task, batch move, upload và worker yêu cầu workload bổ sung được đăng ký trước. “20 VU” là mô hình tải, chưa đồng nghĩa 20 người dùng thật hoặc 20 tài khoản nếu script tái sử dụng token; cách cấp account/token cần được khóa và mô tả riêng.

2.3.4 Noisy neighbor và phép so sánh cặp

Thiết kế cần aggressor và victim, cùng workload và cấu hình giữa hai biến thể limiter tắt/bật. Quan sát cả latency, throughput, error/429 và tài nguyên của victim. Tỷ số p95 victim sau/trước có thể mô tả mức thay đổi khi mẫu số hợp lệ, nhưng không thay dữ liệu gốc hoặc độ biến thiên giữa các cặp run. Thiết kế này xuất phát từ rủi ro tài nguyên chia sẻ; database riêng vẫn có thể chịu ảnh hưởng ở API, CPU hoặc storage [9].

Limiter hiện là in-memory cho một API instance, nên đánh giá multi-instance là câu hỏi khác. Pilot cần tìm tải aggressor gây cạnh tranh nhưng vẫn quan sát được victim; không chọn tải sau khi xem kết quả để ưu ái một placement. Side effect provisioning hoặc SMTP đang chạy trong lúc đo phải được giữ giống nhau hoặc đưa thành biến riêng.

2.3.5 Đơn vị lặp, QA và giới hạn thống kê

Các run độc lập cần reset dữ liệu tương đương, xác nhận cache/warm-up và giữ thứ tự ngẫu nhiên hoặc xen kẽ đã ghi. Ít nhất ba run là điểm xuất phát để quan sát biến thiên, chưa bảo đảm power cho mọi kiểm định. Khi số run nhỏ, ưu tiên mô tả từng run, median/range và paired difference; chỉ dùng khoảng tin cậy hoặc kiểm định sau khi phương pháp và giả định phù hợp đã được khóa.

QA kiểm tra checksum, thời gian cửa sổ, protocol ID, commit/image, seed, metric tags và dữ liệu thiếu. Kết quả không được đưa vào phân tích cuối nếu run_class là

development/pilot, working tree không đáp ứng protocol hoặc eligible flag chưa hợp lệ. Pipeline hiện có được xem là công cụ hỗ trợ kiểm tra; validator pass không thay review raw data.

2.3.6 Đánh giá người dùng và điều kiện thực hiện

Protocol hiện có đề cập tác vụ chuẩn, SUS và góp ý. Nhóm cần xác nhận phạm vi, xin phê duyệt cần thiết, tuyển người tham gia và lấy đồng thuận trước khi thu. Bản sơ bộ không có điểm SUS hoặc tỷ lệ hoàn thành tác vụ. Dữ liệu chỉ nhận mã ngẫu nhiên/đã ẩn danh theo quy trình, và người tham gia được dừng theo điều kiện đồng thuận. Hiệu quả sử dụng và hiệu quả kiến trúc được đánh giá riêng, không lấy sự hài lòng làm chứng cứ rằng không có leak.

Chương 3

Phân tích, lựa chọn và hiện thực kiến trúc

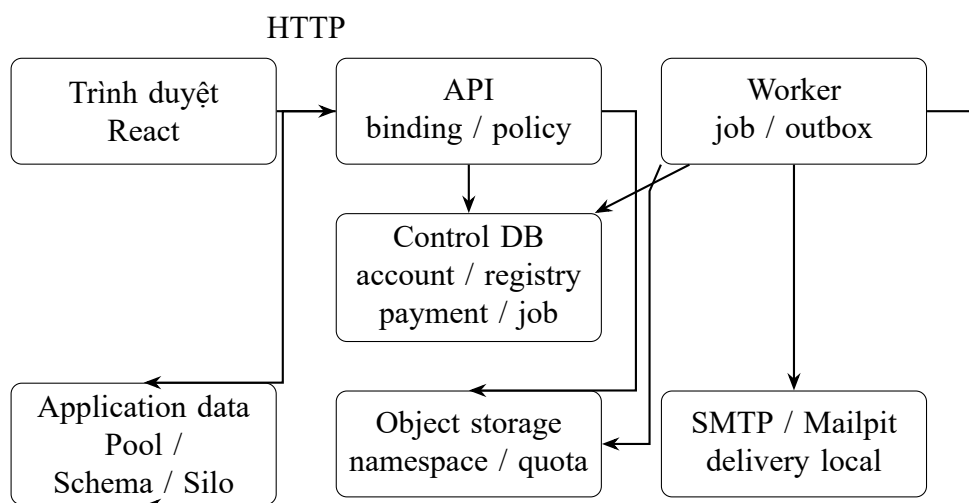
Chương này trình bày hiện thực ở checkout e829f73 và lý do lựa chọn có điều kiện. Mỗi nhóm sử dụng cùng chuỗi bài toán–phương án–tiêu chí–hiện thực–bằng chứng–mục đánh giá. Các phương án chưa cài đặt được dùng để phân tích đánh đổi, không được gọi là ứng viên đã benchmark.

3.1 Kiến trúc tổng thể Bridge

3.1.1 Tổ chức thành phần và hai plane

Ứng dụng dùng một backend Spring Boot với profile API và worker. Frontend React gọi cùng OpenAPI cho các placement. API tiếp nhận request và thực thi nghiệp vụ; worker cấp phát, nâng application schema và xử lý tác vụ nền. PostgreSQL giữ control database và application data; MinIO và Mailpit là adapter dịch vụ trong Compose local. Prometheus/Grafana hỗ trợ quan sát theo cấu hình hiện có E15.

Control plane quản lý account, tenant, membership, tier, placement, route, payment và provisioning. Application plane lưu project, board, task, bình luận, resource, notification và outbox. Hình 3.1 biểu diễn đường truy cập, không phải sơ đồ triển khai đã được nghiệm thu trên Internet.



Hình 3.1: Control/application plane và dịch vụ dùng chung, dựng từ source/contract (E10–E15). Mũi tên thể hiện đường truy cập; không biểu diễn throughput hoặc topology đã nghiệm thu trên Internet.

Một mutation nghiệp vụ dùng một transaction application database qua executor.

Các cập nhật account/payment/job dùng control transaction. Không giả định có ACID transaction xuyên hai plane, storage hoặc SMTP. Workflow cấp phát phối hợp các bước bằng trạng thái, lease và compensation. Thiết kế này giúp ghi rõ điểm có lỗi từng phần thay vì che lỗi bằng một lời gọi “tạo workspace” ở giao diện.

3.1.2 Mô hình dữ liệu và hợp đồng thống nhất

Mỗi bản ghi nghiệp vụ được gắn tenant ID, kể cả ở schema hoặc database riêng. Đây là quy ước hiện thực để giữ cùng SQL, policy, test và audit. Quan hệ project–board–task cùng membership xác định quyền nội dung; quan hệ resource–task cho phép sử dụng lại file/link trong cùng tenant. Soft delete giữ bản ghi phục vụ lịch sử và cleanup, nhưng truy vấn hoạt động cần loại các đối tượng đã xóa.

OpenAPI là nguồn hợp đồng; frontend sinh type từ contract. Request nghiệp vụ mang ID tài nguyên và phiên bản mong đợi, còn tenant identity do host/token đã xác minh quyết định. Tài liệu và diagram phải theo cùng thuật ngữ Pool/Schema-per-tenant/Silo database để tránh gọi schema là database hoặc gọi database riêng là full-stack silo.

3.1.3 Placement registry và vòng đời schema

Registry lưu placement của tenant và metadata đã cấp phát. Resolver chọn datasource từ tenant context; credential đặc quyền chỉ ở đường provisioner. Application migration V1–V12 dùng cho cấu trúc lõi các placement; control plane có V1–V6 ở checkout. Marker schema trong placement giúp worker bỏ qua outbox khi application schema chưa đạt phiên bản mới nhất E11, E14.

Registry còn cho phép mô tả vận hành thống nhất, nhưng không tự chứng minh migration không lỗi trên mọi phiên bản cũ. Bằng chứng upgrade local được ghi theo checkpoint. Không suy rằng mọi database đang chạy hôm nay đều V12 chỉ từ migration có trong source hoặc trạng thái runtime cũ.

3.1.4 Quyết định kiến trúc và trạng thái

ADR-0007 chấp nhận phiên tenant-bound; ADR-0006 chấp nhận state machine và outbox. ADR-0008 chấp nhận Bridge ba placement cho triển khai local, đồng thời ghi phạm vi học thuật chờ xác nhận. ADR-0003/0004/0005 về isolation mechanism, payment và storage vẫn Proposed. Việc RLS, FakePaymentProvider và MinIO xuất hiện trong hệ thống không đồng nghĩa chúng đã thắng so sánh nghiên cứu.

Một điểm khác biệt cần ghi là ADR-0006 mô tả round-robin/bounded concurrency, còn OutboxWorker.poll hiện duyệt các tenant ACTIVE tuần tự bằng stream forEach.

Báo cáo mô tả hành vi code là tuân tự; fairness/bounded concurrency là thiết kế cần kiểm chứng hoặc bổ sung, không phải tính chất đã xác nhận.

3.2 N1 – Định danh, định tuyến và truyền ngữ cảnh

3.2.1 Bài toán và câu hỏi

Một account thuộc A và B đăng nhập ở host A, sau đó gửi access token ấy tới host B. Nếu hệ thống tin Host mà không đối chiếu phiên, request có thể được định tuyến đúng B nhưng quyền lại đến từ A. Nếu chỉ tin token, nội dung A có thể xuất hiện tại địa chỉ B. Sau request, một thread hoặc connection được tái sử dụng cũng có thể giữ context cũ; job nên không còn request để làm nguồn ngữ cảnh.

Câu hỏi N1 là làm thế nào xác lập một tenant đúng, truyền nhất quán tới business SQL/tài nguyên/job và dọn ngữ cảnh khi vòng đời kết thúc. Yêu cầu được đặt trước khi chọn cách định danh; subdomain là tín hiệu định tuyến, chưa là chứng cứ quyền.

3.2.2 Phương án và đánh đổi

Bảng 3.1: So sánh phương án nhận diện tenant

Phương án	Lợi ích trong bối cảnh đề tài	Điều kiện/đánh đổi
Subdomain + token	Workspace có địa chỉ riêng; khớp luồng web hiện có	DNS/proxy/TLS và trusted host; phải đối chiếu quyền
Path/header + token	API thuận tiện, ít phụ thuộc wildcard domain	Input client cần xác minh; giới hạn trình duyệt/proxy với header
Tenant claim trong token	Ngữ cảnh nằm trong token đã ký	Cần tenant switching, revoke và đối chiếu target resource

Các hình thức domain/path/header/claim được Microsoft trình bày như các cách mapping request; bước authorization vẫn cần thực hiện [4]. Trong nguyên mẫu, lựa chọn triển khai là subdomain kết hợp token và lookup hiện hành.

Truyền context bằng tham số làm phụ thuộc rõ và dễ dùng cho async, nhưng tăng số chữ ký hàm. Request scope thuận lợi trong tầng web nhưng không tự có ở worker. ThreadLocal giảm lập tham số trong code đồng bộ, đổi lại phải quản lý set/remove và không tự truyền sang thread khác. Báo cáo đề xuất giữ ThreadLocal hiện tại ở ranh giới đã kiểm soát; khi mở rộng async cần truyền hoặc tái lập context rõ ràng, không kế thừa ngầm.

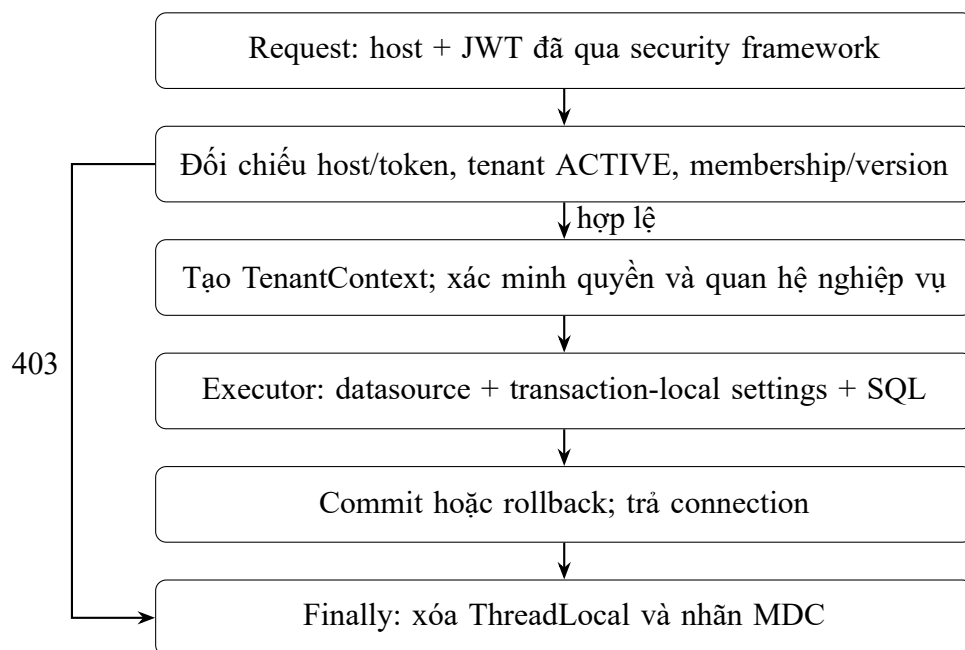
3.2.3 Tiêu chí và lựa chọn

Tiêu chí N1 gồm mismatch bị từ chối trước business service, tenant phải ACTIVE, membership phải hiện hành và context không còn sau request thành công hoặc exception. Worker phải tạo context riêng theo registry và không dùng tenant của vòng poll trước. Tiêu chí chưa bao gồm một cấu hình wildcard DNS/TLS công khai đã được kiểm chứng.

Host-token-status-membership binding đang có phù hợp nguyên mẫu web và ADR-0007. Lookup membership tạo chi phí control plane và ảnh hưởng availability của request; đổi lại server không chỉ tin role cũ trong JWT. Chi phí lookup hoặc cache nếu thêm phải được đo riêng và cache cần tenant/user key cùng invalidation đúng.

3.2.4 Hiện thực tại ranh giới request và SQL

TenantHostResolver đọc serverName, so suffix base domain, loại host trung tâm, prefix trống và nested prefix. TenantContextFilter xử lý JWT tenant kind, so slug host với token, lookup tenant ID/status, active membership và security version, rồi mới tạo TenantContext. Nhánh reject trả 403; finally dọn holder và nhãn MDC. Hình 3.2 mô tả thứ tự kiểm tra E10.



Hình 3.2: Ngưỡng request tới transaction và cleanup theo code hiện tại (E10). Worker tái lập context riêng cho từng tenant.

TenantJdbcExecutor yêu cầu context trước khi lấy connection, tắt auto-commit, đặt app.tenant_id/user_id/correlation_id và search_path bằng set_config có transaction-local flag, thực thi callback rồi commit hoặc rollback. finally khôi phục auto-commit và try-with-resources trả connection. Đây là phạm vi transaction của executor,

không phải một session setting sống mãi trong pool.

OutboxWorker tạo context cho tenant ACTIVE từng lượt, xử lý event trong try/finally và dọn holder/MDC. Việc worker lấy membership để dựng context không cấp quyền tùy ý cho mọi recipient; các handler còn phải kiểm tra event, project và trạng thái người nhận. Nếu code mới gọi SQL ngoài executor hoặc mở async thread, các bất biến này cần được kiểm tra lại.

3.2.5 Bằng chứng, giới hạn và nơi đánh giá

TenantContextFilterTest, TenantHostResolverTest và TenantContextHolderTest chứa tình huống binding và cleanup. E2E lịch sử ghi host/token matrix trên ba placement E05; chưa được gọi là chạy lại hôm nay. Chi tiết kết quả và skip ở Mục 4.2; đường kiểm toán ở E10.

Giới hạn là chưa xác nhận trusted proxy/forwarded header, DNS và TLS trên Internet; context đúng ở unit test không đủ chứng minh cấu hình ingress production. Tiêu chí cần bổ sung khi triển khai là request qua reverse proxy thật, spoofed forwarding input và context cleanup khi error path chạy xuyên tầng.

3.3 N2 – Bố trí và cô lập dữ liệu

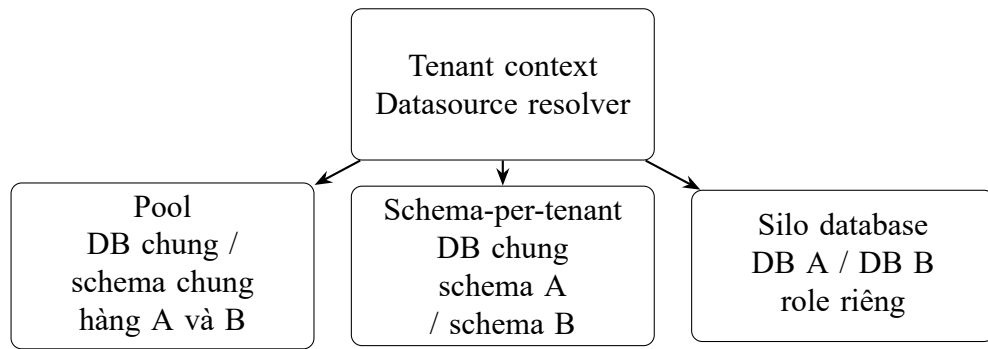
3.3.1 Bài toán và hai cấp quyết định

Người dùng A biết ID task/resource của B; một query native hoặc bulk thiếu tenant predicate; một connection được trả pool sau transaction lỗi. N2 hỏi cách tổ chức dữ liệu và enforcement phù hợp cho Kanban trên hạ tầng giới hạn. Bố trí dữ liệu được quyết định ở cấp topology; predicate/ORM/RLS được phân tích ở cấp cơ chế. Hai cấp có thể phối hợp thay vì loại trừ nhau.

3.3.2 So sánh bố trí dữ liệu

Pool tập trung migration và tận dụng cấu trúc bảng chung, nhưng phụ thuộc tenant enforcement ở mọi bảng/đường truy cập và chịu cạnh tranh tài nguyên database. Schema-per-tenant có namespace và role riêng, đổi lại nhiều schema/migration; database và host vẫn dùng chung. Silo database có biên quản trị/backup riêng ở cấp database, đổi lại tăng pool, provisioning và migration theo tenant. Các nhận xét này là phân tích cấu trúc; chưa là số đo chi phí hoặc hiệu năng.

Trong Bridge hiện tại, cùng API và migration nghiệp vụ giúp hạn chế khác biệt chức năng khi so sánh. Placement được chốt trong onboarding và không di chuyển khi



Cùng API và migration nghiệp vụ; control plane, API, worker và tài nguyên máy chủ có thể vẫn dùng chung.

Hình 3.3: Ba topology application data trong Bridge (E11). Schema là namespace; quyền role và policy bổ sung enforcement.

đã có dữ liệu. Việc chọn placement cho một tenant vì vậy có chi phí vòng đời, không chỉ chi phí một query. Bộ bảng giữ tenant_id trên cả ba placement để duy trì contract và các guard chung E11.

3.3.3 So sánh enforcement trong Pool

Bảng 3.2: Các cơ chế bảo vệ Pool và đường kiểm chứng

Cơ chế	Lợi ích	Failure mode cần kiểm tra
Predicate tường minh	Điều kiện nhìn thấy ngay trong SQL/repository	Query mới, bulk hoặc background bỏ sót điều kiện
Tenant/filter của ORM	Tập trung một phần enforcement trong ORM	Native SQL, bulk và đường truy cập ngoài ORM
RLS + application guard	Policy database bổ sung bảo vệ ở đường SQL	Runtime role, owner, context và policy cấu hình sai

Đề xuất hiện tại là tiếp tục RLS cộng guard ứng dụng để bảo vệ nhiều lớp. Đây là lựa chọn triển khai có bằng chứng sàng lọc local; ADR-0003 vẫn Proposed vì hồ sơ loại, measurement và scorecard chưa được review đầy đủ. Chưa đề xuất placement nào thắng mọi tiêu chí.

3.3.4 Tiêu chí kiểm chứng

Đọc/ghi/xóa ID của B từ A phải bị chặn và trạng thái B giữ nguyên. Native/bulk/background đều cần negative case. Connection sau commit/rollback không mang app.tenant_id hoặc search_path cũ sang transaction khác. Runtime role không phải superuser hoặc BYPASSRLS; ownership, schema privileges, default privileges và

quyền database phải được đối chiếu riêng. Các đường metadata/storage cần guard tương ứng vì RLS chỉ tác động SQL của bảng được bảo vệ.

PostgreSQL ghi rõ superuser và BYPASSRLS có thể vượt policy; owner thường vượt nếu không FORCE RLS. search_path chỉ chọn namespace và không thay quyền schema [10], [11]. Vì vậy policy bật trong migration chưa đủ nếu application credential có quyền vượt biên.

3.3.5 Hiện thực resolver và runtime role

DefaultTenantDataSourceResolver trả datasource Pool chung hoặc tạo/cache datasource cô lập theo tenant ID cho Schema/Silo. Resolver kiểm tra placement, metadata database/credential và schema name hợp lệ. Với Schema-per-tenant, runtime role/schema riêng và search_path transaction-local chọn đúng namespace. Với Silo, runtime role kết nối database riêng. Tên vật lý và credential đến từ registry đã provision, không lấy từ payload nghiệp vụ.

TenantDatabaseProvisioner chuẩn bị tài nguyên bằng credential đặc quyền, áp dụng migration và quyền; API dùng role runtime. Application SQL qua executor vẫn chứa guard tenant ở nhiều query. RLS cộng predicate không phải hai cơ chế thừa nhau: một lớp hỗ trợ authorize quan hệ nghiệp vụ, lớp còn lại hạn chế lỗi bỏ sót query.

3.3.6 Bằng chứng sàng lọc và diễn giải đúng

Harness guarded contract lịch sử pass 6/6 cho ba cơ chế trên Pool/Silo. Khi có ý bỏ guard ở native/bulk/background, hồ sơ ghi 18 quan sát: sáu leak của explicit/Hibernate Pool, ba RLS Pool được bảo vệ và chín đường Silo được biên database bảo vệ E09. Harness pass nghĩa là assertion phân loại đúng; sáu leak là hành vi không an toàn của ứng viên, không phải sáu kiểm tra bảo mật đạt.

RowLevelSecurityTest kiểm tra policy/runtime role; SchemaPerTenantIsolationIntegrationTest chứa truy vấn schema khác, rollback và cleanup tenant A giữ B. EXT ghi kết quả schema isolation trên PostgreSQL thật tại mốc 08/09 E05. Kết quả/skip hiện có được giới hạn ở Mục 4.2. Không lấy test của owner/superuser làm bằng chứng runtime role đã an toàn.

3.3.7 Giới hạn và công việc còn thiếu

Chưa có phép đo latency/RAM/connections của các ứng viên còn lại, chưa có hồ sơ loại checksum-backed được review và chưa khóa comparison ba placement trong protocol. Các kiểm tra RLS không bao phủ mọi policy mới nếu thêm bảng, và biên database

không giải quyết quyền project hoặc storage. Chi phí migration, restore từng tenant, cache datasource và ảnh hưởng noisy neighbor cần đánh giá riêng trước khi trả lời RQ4 cuối cùng.

3.4 N3 – Xác thực và phân quyền tenant/project

3.4.1 Bài toán và câu hỏi

Owner tenant gọi API đọc project mà họ không tham gia; Member gọi trực tiếp API quản trị; người đã bị revoke vẫn giữ JWT còn hạn. N3 hỏi cách tách danh tính, phiên tenant và quyền theo phạm vi để các thao tác phản ánh membership hiện hành. Sự hiện diện của nút UI không quyết định quyền server.

3.4.2 Phương án và đánh đổi

Bảng 3.3: So sánh nguồn quyết định quyền

Phương án	Lợi ích	Điều kiện/đánh đổi
Role chủ yếu trong JWT	Ít lookup tại request	Quyền cũ tồn tại tới expiry nếu không có invalidation
Membership server hiện hành	Thu hồi và xét quyền theo scope	Lookup và phụ thuộc database; cache cần invalidation
ABAC/ReBAC policy	Diễn đạt điều kiện/quan hệ phức tạp	Thêm mô hình policy và khối lượng kiểm chứng

Ở mức identity, ứng dụng tự quản lý account cho phép nguyên mẫu local hoạt động ít phụ thuộc dịch vụ ngoài. IdP/OIDC có thể tập trung federation/lifecycle nhưng cần tích hợp tenant mapping và project policy của ứng dụng. Tài liệu Microsoft mô tả các quyết định identity trong hệ thống đa tenant [12]. Báo cáo so sánh ở mức kiến trúc; không chuyển hệ thống sang IdP trong công việc này.

3.4.3 Tiêu chí và lựa chọn triển khai

Tiêu chí là ma trận allow/deny theo hành động, direct API không vượt role, Owner thiếu project membership không đọc nội dung, token cũ bị chặn sau membership thay đổi và deny không tạo side effect. Role ở một project không được cấp quyền project khác. Tenant suspended cần bị chặn trước business data; archived project có thể đọc nhưng mutation phải bị từ chối theo invariant riêng.

Lựa chọn là account chung, tenant-bound session, membership hiện hành và RBAC tenant/project. Cách này giữ mô hình nghiệp vụ đơn giản và thu hồi quyền theo phiên bản security. ABAC/ReBAC chưa cần được triển khai đầy đủ cho tập hành động hiện tại, nhưng một số điều kiện như tác giả comment/uploader là policy bổ sung theo đối tượng, không thể chỉ xét cấp role.

3.4.4 Hiện thực identity và phiên tenant

AuthService xử lý account login, transfer code và tenant session. Transfer code có scope và vòng đời riêng; frontend exchange để nhận access token tenant và refresh flow. Backend xác minh token bằng security framework; filter kiểm tra lại tenant/membership/version. Access token giữ trong memory frontend; logic refresh và tenant switching cần bảo toàn ranh giới cache.

Một phiên đã có tenant claim vẫn có thể bị revoke qua membership/security version. Điều này tách hiệu lực role hiện hành khỏi thời hạn JWT. Nếu thêm cache membership, thời điểm invalidation sẽ trở thành một phần tiêu chí chứ không được bỏ qua bằng giả định token đã ký là đủ.

3.4.5 Hiện thực quyền nội dung

ProjectApplicationService tìm quan hệ tài nguyên và gọi requireProjectRole trong callback executor trước mutation. Role Manager/Member/Viewer có phạm vi project; resource service kiểm tra project relation và ownership theo policy. Invariant Manager cuối cùng ngăn một project mất người quản lý do revoke. Các điều kiện này được kiểm tra tại server, còn UI dùng quyền để hiển thị luồng phù hợp.

Quyền hiệu lực với nội dung là giao của tenant ACTIVE, active tenant membership, host/token binding và project action policy. Capability mở rộng là điều kiện bổ sung, không thay role. System Admin quản trị metadata vận hành không mặc nhiên đọc mọi project. Ma trận quyền rút gọn và đường truy về policy đầy đủ ở Phụ lục B.1.

3.4.6 Bằng chứng và giới hạn

ProjectAuthorizationIntegrationTest có case cross-tenant ID giữ nguyên dữ liệu ngoài phạm vi, deny theo role, project archived, task batch và resource. Security tests kiểm tra membership revoke/version; E2E lịch sử có Manager/Member/Viewer và API trực tiếp E05, E12. Coverage được xét theo assertion và lần chạy ở Mục 4.2, không theo số nút UI đã có.

Chưa kết luận nguyên mẫu là hệ thống identity production được chứng nhận, chưa

có dữ liệu federation/SSO hoặc proof cho mọi policy tổ hợp. Để đóng kết luận cần chạy tích hợp hiện tại, rà từng ô permission matrix với route/handler và bổ sung negative case khi thêm hành động. Việc từ chối nên được kiểm tra cả mã phản hồi lẫn dữ liệu/event trước–sau.

3.5 N4 – Nhất quán trạng thái và xử lý nền

3.5.1 Bài toán và câu hỏi

Hai thành viên mở cùng task tại version v, rồi cùng lưu thay đổi. Một batch move chứa nhiều task nhưng một version đã cũ. Sau mutation, event có thể được worker đọc lại; deadline hoặc assignee có thể đổi trước khi notification được gửi. N4 hỏi cách tránh ghi đè âm thầm, giữ batch nguyên tử và bảo đảm side effect vẫn phù hợp trạng thái hiện hành.

Các tình huống được chia thành hai ranh giới: cập nhật trong application transaction và xử lý event sau transaction. Một khóa hoặc version ở task không tự giải quyết delivery lặp; một outbox không tự giải quyết conflict giữa hai client. Thiết kế cần cơ chế riêng và tiêu chí riêng cho từng ranh giới.

3.5.2 Phương án cập nhật đồng thời

Bảng 3.4: So sánh cách xử lý cập nhật đồng thời

Phương án	Lợi ích	Đánh đổi
Ghi sau cùng thắng	Luồng cập nhật đơn giản	Có thể mất thay đổi mà client không biết
Optimistic version	Phát hiện dữ liệu client đã cũ	Client phải giữ input và xử lý conflict
Pessimistic locking	Điều phối tại transaction	Chờ khóa, transaction dài và nguy cơ deadlock

Nguyên mẫu dùng optimistic version cho task/board và transaction cho batch. Lựa chọn phù hợp thao tác tương tác có thể tải lại khi conflict, nhưng chưa được benchmark với tần suất cạnh tranh cao. Pessimistic lock vẫn có thể dùng cho invariant hoặc workflow cụ thể; không xem hai cách là bắt buộc loại trừ trong toàn hệ thống.

3.5.3 Phương án chuyển side effect

Gửi notification trực tiếp sau mutation có ít thành phần nhưng tạo khoảng lỗi khi dữ liệu đã commit mà delivery thất bại. Gửi trước commit có thể báo một thay đổi sau đó rollback. Transactional outbox ghi event cùng mutation, để worker retry; đổi lại có độ trễ nền, retention và deduplication. Broker có thể hỗ trợ phân phối khi quy mô tăng nhưng thêm thành phần vận hành và vẫn cần giải quyết quan hệ commit-publish.

Lựa chọn hiện tại là PostgreSQL outbox trong cùng application transaction, không thêm broker ở nguyên mẫu. Handler kiểm tra lại tenant, event và recipient trước side effect. In-app/email delivery có sổ và key chống trùng theo hiện thực. Không gọi thiết kế này là exactly-once toàn đường SMTP.

3.5.4 Tiêu chí kiểm chứng

Một expected version cũ phải trả conflict, không ghi đè silent. Khi một item batch thất bại, trạng thái tất cả task và event của batch phải rollback. Một event được dispatch hai lần không tạo notification nghiệp vụ trùng trong phạm vi unique key đã kiểm tra. Nhắc hạn phải loại nếu deadline, assignee, membership hoặc trạng thái task/project không còn phù hợp; không gửi cho người nhận cũ chỉ vì họ có mặt trong payload lịch sử.

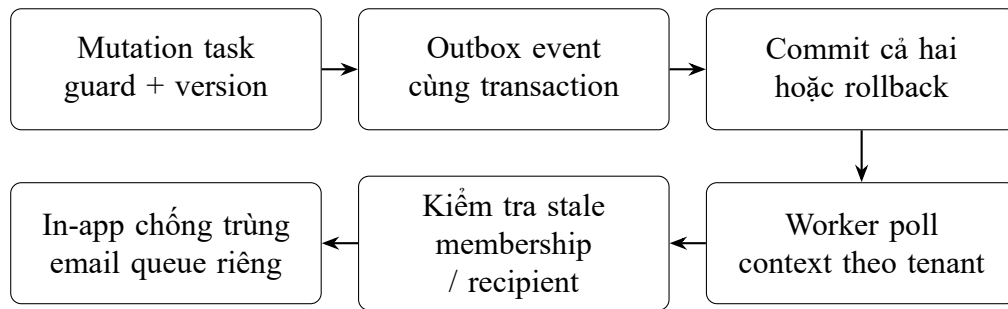
Tiêu chí delivery cần tách enqueue, in-app persisted, email queued và SMTP sent. Một hàng email PENDING chưa là email đã gửi. Một lần gọi SMTP thành công cũng chưa chứng minh người dùng đã đọc hoặc provider ngoài local có SLA.

3.5.5 Hiện thực mutation và outbox

ProjectApplicationService cập nhật task với điều kiện tenant/ID/version, tăng version và báo conflict khi không đủ điều kiện. Batch dùng cùng executor transaction, kiểm tra board/column và version của từng task; event TASK_MOVED được tạo cho từng task trong cùng transaction. Semantic completion của column giúp trạng thái hoàn tất không phụ thuộc tên cột Done hoặc Hoàn tất E13.

OutboxWorker duyệt tenant ACTIVE tuần tự, bỏ qua placement chưa có schema version mới nhất, đặt context riêng, đọc event sẵn sàng theo tenant và retry hữu hạn. Code pendingEvents chưa thể được mô tả như một cơ chế claim SKIP LOCKED đa-worker tương đương provisioning. Deduplication trong handler/delivery phải được đánh giá trên chính đường side effect, không kế thừa kết luận từ job claimer khác.

DeadlineReminderService tạo event due/overdue theo điều kiện task/column/project/membership. Application V11 lưu sổ chống trùng, V12 thêm action URL và email delivery retry. Worker kiểm tra lại trạng thái trước dispatch; email có hàng đợi



Event lặp cần deduplication; event stale bị loại. Retry email không chạy lại mutation. Chưa bảo đảm exactly-once SMTP.

Hình 3.4: Mutation–outbox–delivery theo E13 và hồ sơ nhắc hạn E06. Đây là sơ đồ luồng, không phải số đo độ trễ worker.

riêng, backoff/lease và số lần thử tối đa năm. NotificationDeliveryWorker tôn trọng preference hiện hành; lỗi SMTP không phải lý do để xử lý lại toàn bộ mutation Kanban.

3.5.6 Bằng chứng và giới hạn

ProjectAuthorizationIntegrationTest có taskBatchReorderIsAtomicAndManagerSoftDeleteIncludesOneLevelSubtasks. DeadlineReminderIntegrationTest có case enqueue idempotent, deadline thay đổi, exact assignee, reminder stale và dispatcher tạo in-app/email queue chống trùng. Biên bản nhắc hạn ghi 6/6 integration test trên PostgreSQL cùng application V1–V12 E06; lần test hiện tại skip các case Docker E04.

Kết luận sơ bộ là code có cơ chế conflict, transaction và kiểm tra stale với hồ sơ local hỗ trợ các case đã ghi. Chưa có kiểm chứng nhiều outbox worker đồng thời, fairness giữa tenant hoặc exactly-once SMTP; chưa có realtime collaboration. Mục 4.2 tổng hợp bằng chứng, Mục 4.3.2 thảo luận giới hạn và điều kiện kiểm tra tiếp.

3.6 N5 – Cấp phát và quản lý tài nguyên dùng chung

3.6.1 Bài toán và câu hỏi

Callback thanh toán lập có thể tạo nhiều job; worker chết sau DDL nhưng trước finalize; migration lỗi để lại tài nguyên một phần. Tenant tải cao có thể chiếm connection, CPU hoặc storage và ảnh hưởng tenant khác. N5 hỏi cách quản lý vòng đời tài nguyên có thể phục hồi và giới hạn chia sẻ ở quy mô mục tiêu 3–5 tenant.

Đúng trạng thái provisioning và công bằng tài nguyên là hai mục tiêu liên quan nhưng có phép đánh giá khác nhau. Một tenant ACTIVE sau migration không chứng

minh khả năng chịu tải; một quota tồn tại trong code không chứng minh mọi cuộc đua đa-instance được xử lý đúng.

3.6.2 Phương án cấp phát và lựa chọn

Bảng 3.5: So sánh phương án cấp phát tenant

Phương án	Lợi ích	Đánh đổi
Cấp phát thủ công	Ít thành phần tự động	Lỗi thao tác, khó tái lập/retry/audit
Đồng bộ trong request	Luồng dễ theo dõi	Request dài và lỗi từng phần sau side effect
Worker/state machine/lease	Claim, checkpoint, retry và recovery rõ	Thêm trạng thái, idempotency và compensation

Lựa chọn hiện thực là worker/state machine với credential đặc quyền riêng. API chỉ ghi yêu cầu/control metadata; worker tạo database/schema/role, chạy migration và hoàn tất registry. FakePaymentProvider phục vụ local/test, không chứng minh adapter VNPay/Stripe thật đã đạt signature/callback contract. ADR payment còn Proposed và cần credential sandbox cùng scorecard.

3.6.3 Tiêu chí cấp phát và phục hồi

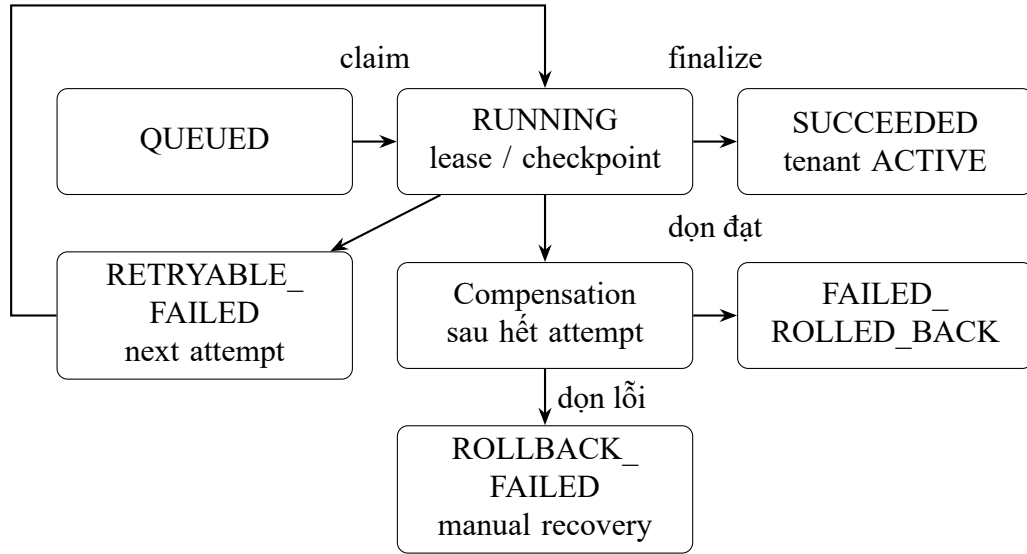
Callback hoặc retry trùng không tạo thêm job/placement/role/database ngoài tài nguyên đã định danh. Hai worker claim cùng job chỉ một claim hợp lệ; worker mất lease không được finalize hoặc đổi trạng thái bằng token cũ. Tenant chỉ ACTIVE sau khi tài nguyên, schema và route đủ điều kiện. Rollback chỉ dọn tài nguyên đúng ownership; khi dọn thất bại phải giữ trạng thái lỗi có thể audit/manual recovery, không báo sạch giả.

Các ranh giới crash cần kiểm tra gồm sau DDL, sau Flyway và trước finalize. Recovery phải dùng metadata/credential ổn định đã chuẩn bị, không sinh database mới ở mỗi attempt. Giới hạn này quan trọng vì DDL và control update không nằm trong cùng một transaction.

3.6.4 Hiện thực state machine và lease

ProvisioningJobClaimer dùng transaction ngăn, row locking và SKIP LOCKED để ghi lease owner/token/expiry. ProvisioningJobCoordinator kiểm tra ownership và attempt khi prepare/finalize/failure; DefaultProvisioningService thực hiện các

bước ngoài control transaction dài. TenantDatabaseProvisioner áp dụng DDL, migration và quyền runtime. Hình 3.5 mô tả các trạng thái và nhánh lỗi E14.



Hình 3.5: State machine provisioning rút gọn theo coordinator (E14). Retry/manual recovery chỉ qua transition hợp lệ; finalize kiểm tra lease ownership.

Trạng thái SUCCEEDED đi cùng việc tenant ACTIVE ở finalize hợp lệ. Lỗi còn attempt được ghi RETRYABLE_FAILED và có next-attempt. Sau giới hạn thử, compensation thành công dẫn tới FAILED_ROLLED_BACK; compensation thất bại là ROLLBACK_FAILED. Manual retry được API/state machine giới hạn và ghi audit. Các tên trạng thái của job không được dùng thay trạng thái tenant khi đọc status ở giao diện.

3.6.5 Connection pool và ngân sách tài nguyên

Resolver giữ một Pool datasource và cache datasource Schema/Silo theo tenant. Hikari có minimumIdle bằng 0, maximumPoolSize và timeout cấu hình; datasource nhận rồi được eviction định kỳ. Ngân sách Schema và Silo được cấu hình riêng. Trước khi đưa vào phép đo, cần cộng cả control pool, API/worker instance, Pool datasource, datasource cô lập, migration và connection giám sát.

Một biểu thức lập kế hoạch cho process là

$$C_{\text{plan}} = C_{\text{control}} + C_{\text{pool}} + n_{\text{schema}}p_{\text{schema}} + n_{\text{silo}}p_{\text{silo}} + C_{\text{other}}. \quad (3.1)$$

Các p là maximumPoolSize và n là số datasource hoạt động trong process; đây là ngân sách dự kiến, không phải số connections đo được. API và worker có thể có cache riêng. enforceGlobalCap hiện kiểm tra số datasource/cache theo placement để eviction; báo cáo không coi logic ấy là bằng chứng hard bound nguyên tử trước mọi cuộc đua tạo pool

đa-thread hoặc đa-process. Cần đo và kiểm tra concurrency trước khi kết luận trên toàn hệ thống.

3.6.6 Storage, quota và cleanup

ResourceService kiểm tra role, namespace và quota trước upload. Object key sinh theo tenant/resource; ResourceStorageKey và adapter từ chối key ngoài phạm vi. File có metadata và object; link HTTP(S) là tài nguyên không có object để dọn. Xóa mềm và physical deletion được tách qua outbox, với retry/dead-letter/requeue có audit. Storage SPI cho phép so filesystem và MinIO, nhưng ADR-0005 chưa có số đo/scorecard để chốt bên thắng.

Quota upload trong code hiện được tuần tự hóa bằng ReentrantLock theo tenant trong một API process. Khóa này không tự bảo vệ nhiều API instance. Store object rồi ghi metadata cũng có khoảng lỗi và bước compensation nếu database write thất bại; cần fault case cho compensation. Signed URL có thời hạn là một capability đã cấp, nên revoke membership không mặc nhiên thu hồi mọi URL còn hiệu lực. Giới hạn này phải được mô tả khi đánh giá policy download.

3.6.7 Limiter và quan sát noisy neighbor

TenantRateLimitFilter dùng bucket in-memory keyed tenant ID, capacity theo tier và trả HTTP 429/Retry-After khi không còn token. Cơ chế phù hợp phạm vi một API instance, chưa có đồng bộ nhiều instance hoặc eviction bucket dài hạn. Việc có limiter chưa chứng minh victim được bảo vệ dưới mọi workload; mục đo còn thiếu là paired aggressor/victim trước–sau limiter, cùng seed và cấu hình.

Metric/log cần tenant labels để phát hiện ảnh hưởng tới từng tenant, đồng thời tránh token hoặc nội dung nghiệp vụ. Prometheus hiện scrape API; worker không có endpoint metric riêng theo hồ sơ. Noisy neighbor có thể xuất phát ở API, PostgreSQL host, object storage hoặc nền, nên database Silo không loại bỏ mọi tài nguyên dùng chung.

3.6.8 Bảng chứng, giới hạn và nơi đánh giá

Hồ sơ P1 ghi claim, crash JVM con ở ba ranh giới, recovery và rollback PostgreSQL lỗi lặp rồi manual recovery E08. Với MinIO outage, Pool/Silo tới dead letter lần thứ năm; requeue từng tenant giữ nguyên tài nguyên tenant còn lại. Đây là kết quả local lịch sử; lượt này không chạy lại fault injection.

Các case tích hợp và giới hạn coverage xem Mục 4.2; thiết kế phép đo xem Mục 4.3. Connection budget, hiệu quả limiter và provider Internet còn PENDING_DATA.

Chương 4

Bằng chứng kỹ thuật, kế hoạch thực nghiệm và bàn luận

Chương này báo cáo đúng loại bằng chứng đang có và mức trả lời RQ. Các bảng kết quả chưa có dữ liệu được giữ như mẫu thu thập với trạng thái rõ ràng, không điền số minh họa.

4.1 Baseline và hồ sơ kiểm chứng

4.1.1 Phiên bản mô tả và môi trường viết

Nền source là e829f73, nhánh main; nguồn LaTeX của bản sơ bộ còn trong working tree. Bảng migration ở source gồm control V1–V6 và application V1–V12. Đây là mô tả code, không phải truy vấn trạng thái một stack đang chạy. Hồ sơ E01 lưu khác biệt giữa checkout và checkpoint cũ của PROJECT_STATUS.

Môi trường xuất bản có Python 3.12.3, XeTeX/TeX Live 2023, Biber và Poppler. Doctor xác nhận các package và đủ bốn kiểu Times New Roman. Maven hiện có là 3.9.11. Node.js Linux chưa dùng được; npm từ Windows báo không xác định được Node directory. Docker-dependent suite không có runtime Docker để thực thi trong lượt kiểm tra. Chi tiết backend thực tế và compiler được ghi trong biên bản mới E04.

4.1.2 Các checkpoint lịch sử được giữ riêng

Bảng 4.1: Checkpoint kỹ thuật và giới hạn sử dụng

Mốc / evidence	Nội dung được ghi trong hồ sơ	Phạm vi kết luận
P0 26/08; E07	Baseline Pool/Silo, migration, role/RLS và smoke local	Lịch sử khởi chạy; timing smoke không là số đo performance
P1 cuối 08/2026; E08	Claim/lease/crash recovery, rollback lỗi, storage cleanup	Tình huống và môi trường local; không tự áp dụng cho production
P2 local 31/08; E09	Guarded 6/6; 18 guard-omission observations, gồm 6 leak	Sàng lọc ứng viên; measurement/scorecard chưa có

Mốc / evidence	Nội dung được ghi trong hồ sơ	Phạm vi kết luận
EXT 08/09; E05	Application V8; backend 96/96, skip 0; frontend 16; E2E 3	Ba placement và tùy biến local tại mốc đó
Nhắc hạn 19/09; E06	Application V12; backend 106; frontend 22; E2E 3 theo biên bản	File mang tên 18/09; ngày nội dung 19/09; không là lượt chạy hôm nay

Đối chiếu số test hoặc migration theo mốc giúp tránh cộng dồn hoặc thay thế lịch sử. Một test đã pass ở V8 không đủ chứng minh mọi thay đổi V12 đã được chạy trong môi trường hiện tại.

4.1.3 Artifact cũ và kiểm tra mới

Artifact Surefire có sẵn ghi 106 test, 0 failure/error, 47 skip vì Docker không khả dụng: 59 test được thực thi E03. Commit thực thi chưa được xác lập. Lượt mới biên dịch lại 121 source và 27 test source, rồi mục tiêu test đạt: 106 tổng, 59 thực thi, 47 skip, 0 failure/error E04. Maven và test runtime là Java 21.0.12.1; compiler fork là javac 25.0.4.1 với release 21 vì compiler đi kèm runtime 21 trên máy báo không hỗ trợ release 21. Đây không phải build hoàn toàn bằng JDK 21.

Mục tiêu verify chưa hoàn tất do thiếu plugin trong cache offline. Frontend/E2E chưa chạy lại vì thiếu Node/runtime stack. Không nâng nhãn “test goal đạt” thành “verify đạt”, không suy 47 case Docker bị skip là pass và không dùng compiler fallback làm bằng chứng đã đạt môi trường verification chuẩn.

Các PDF sinh kèm được kiểm tra theo cùng trạng thái sơ bộ. Build và check xác nhận nguồn có thể xuất bản, A4, phong nhúng và tham chiếu ổn định; không xác nhận protocol đã được duyệt, ADR đã chốt hoặc nghiệm thu nghiên cứu. Kết quả cuối của build được lưu trong report/PRELIMINARY_VALIDATION.md.

4.2 Bảng chứng theo nhóm và câu trả lời RQ sơ bộ

4.2.1 Phạm vi coverage cần được diễn giải

Bảng 4.2 liên kết nhóm với assertion quan trọng. Cột evidence phân biệt source inspection, checkpoint lịch sử và kết quả mới. Số test không là coverage percentage:

nhiều assertion ở một test có thể kiểm tra nhiều hành động, trong khi một nhánh chưa có negative case vẫn có thể bị bỏ sót.

Bảng 4.2: Coverage, bằng chứng và phần chưa đóng

Nhóm	Tình huống/tiêu chí cần truy vết	Bằng chứng hiện có	Giới hạn còn lại
N1	Token A/host B; revoke/version; tenant suspended; cleanup exception	Filter/holder/host tests; E10; E04 cho test thực thi	Chưa xác nhận ingress Internet và trusted proxy
N2	IDOR read/write/delete; native/bulk/background; explicit schema B; commit/rollback reuse	E09 screening; RLS/schema tests; E11; EXT E05	RLS 4 và schema 1 test Docker bị skip; chưa có measurement
N3	Role allow/deny; Owner thiếu project role; revoke; deny không side effect	Project authorization 17 case; security tests; E12	17 integration case bị skip; chưa đóng mọi ô permission matrix hiện tại
N4	Stale version; batch rollback; duplicate/stale event; exact assignee	Source E13; deadline 6 case, E06	6 deadline integration case bị skip; multi-worker chưa kiểm chứng
N5	Duplicate callback; atomic claim; crash/lease; compensation; storage dead letter	E08; provision/storage source E14	Integration Docker bị skip; chưa có budget/limiter đo thật hay provider Internet

4.2.2 Số test bị bỏ qua và hệ quả

Trong bộ Surefire 106 test, nhóm Docker gồm ProjectAuthorizationIntegrationTest 17, CustomizationIntegrationTest 3, RowLevelSecurityTest 4, DeadlineReminderIntegrationTest 6, PaymentConcurrencyIntegrationTest 2, ProvisioningJobClaimerIntegrationTest 4, ProvisioningJobCoordinatorIntegrationTest 4, SchemaPerTenantIsolationIntegrationTest 1, TenantDatabaseProvisionerFailureInjectionIntegrationTest 1 và ResourceDeletionOutboxIntegrationTest 5. Tổng là 47 case. Không có failure/

error trong các case thực thi không xóa được giới hạn của 47 case bị skip.

Các test unit/security thực thi giúp kiểm tra luật binding, management, payment và adapter trong môi trường không Docker. Chúng không thay policy SQL, migration thật, MinIO thật hoặc crash PostgreSQL thật. Hồ sơ lịch sử vẫn cung cấp bối cảnh cho các tình huống đó, nhưng cần tái kiểm chứng trên phiên bản khóa trước kết luận tích hợp cuối.

4.2.3 Diễn giải sàng lọc isolation

Bảng 4.3: Guard-omission observations lịch sử, không phải benchmark

Ứng viên/placement	Số quan sát	Kết quả ghi trong hồ sơ E09
Explicit predicate / Pool	3	Leak khi bỏ guard ở native, bulk và background
Hibernate mechanism / Pool	3	Leak ở cùng ba đường deliberate omission
RLS + guard / Pool	3	Được policy bảo vệ trong các case omission
Cả ba ứng viên / Silo	9	Biên database bảo vệ trong các case được kiểm tra

Điều kiện loại đã đăng ký xem mọi cross-tenant success là hành vi loại. Hai ứng viên application-only vì vậy có dấu hiệu loại ở Pool. Bản sơ bộ không tự hoàn tất hồ sơ loại checksum-backed hoặc chấm công nghệ còn lại; trạng thái nghiên cứu P2 measurement vẫn tạm dừng. Chín observation Silo không chứng minh mọi thao tác storage/authorization hoặc mọi cấu hình database đều an toàn.

4.2.4 RQ1 – Yêu cầu nào cần cho ứng dụng?

Trả lời sơ bộ, INFERRED: hồ sơ yêu cầu và nguyên mẫu cho thấy cần account chung, workspace có scope, tenant/project membership, Kanban có version, resource và notification đúng tenant, onboarding có recovery và metric theo tenant. Cơ sở là SRS/permission matrix/OpenAPI E01, E15, cùng các tình huống N1–N5. **Giới hạn:** chưa có user study/extraction khảo sát đầy đủ để xác nhận ưu tiên hoặc mức đáp ứng nhu cầu người dùng. **Cần bổ sung:** review yêu cầu với nhóm/người hướng dẫn và thu dữ liệu người dùng nếu được xác nhận trong phạm vi.

4.2.5 RQ2 – Xây dựng Bridge dùng chung thành phần thể nào?

Trả lời sơ bộ, INFERRED từ hiện thực: tách control/application plane, giữ cùng API/mã nghiệp vụ/migration, chọn datasource qua verified tenant context và quản lý provisioning bằng registry/state machine. Code đã mở rộng ba placement; baseline RQ Pool/Silo vẫn được giữ và extension nhận nhãn phạm vi bổ sung. **Bằng chứng:** E10–E15, EXT E05. **Giới hạn:** chưa đóng tích hợp hiện tại vì skip, chưa kiểm chứng di chuyển tenant có dữ liệu hoặc vận hành Internet. **Cần bổ sung:** suite Docker cùng commit và phụ lục protocol cho ba placement.

4.2.6 RQ3 – Kiến trúc đáp ứng đến mức nào?

Trả lời sơ bộ: source và biên bản hỗ trợ cơ chế binding, isolation, role, conflict, dedupe và recovery trong các case local được ghi. Các kết quả lịch sử không thay lần chạy hiện tại; skip được giữ trong đánh giá. **Bằng chứng:** E04–E14. **Phần chưa trả lời, PENDING_DATA:** hiệu năng, efficacy noisy-neighbor limiter, khả dụng trên VPS và phạm vi nhiều worker. **Cần bổ sung:** kiểm chứng tích hợp khóa phiên bản, pilot/run độc lập, fault model vận hành và QA metric. Chưa kết luận zero-leak toàn hệ thống hoặc đạt SLO production.

4.2.7 RQ4 – Cơ chế/công nghệ nào phù hợp nhất?

Trả lời sơ bộ, INFERRED có điều kiện: host/token binding, membership hiện hành, optimistic version, outbox và state machine là lựa chọn hiện thực có lý do kỹ thuật. RLS + guard có căn cứ sàng lọc local cho Pool; adapter tách payment/storage để duy trì lựa chọn. **Bằng chứng:** so sánh N1–N5, ADR và E09. **Giới hạn:** chưa có scorecard/raw measurement đủ để xác định placement hoặc provider phù hợp nhất trên VPS. **Cần bổ sung:** hồ sơ loại review, phép đo tài nguyên/hiệu năng và ADR được chấp nhận rõ ràng.

4.3 Kế hoạch thu số liệu và bàn luận giới hạn

4.3.1 Bảng kết quả cần thu

Các bảng dưới đây định nghĩa đầu ra cho đợt sau; không chứa run thực nghiệm đã thực hiện. Khi có dữ liệu, mỗi hàng phải liên kết run ID, checksum và manifest. So sánh ba placement cần phụ lục protocol; các giá trị warm-up/thời gian đo/lặp ở Chương 2 là đề xuất cần khóa trước.

Bảng 4.4: Trạng thái kết quả performance chưa có dữ liệu

Placement	Tổ hợp dự kiến	Đầu ra cần thu	Trạng thái
Pool	3/5 tenant; 10/20 VU	p50/p95, throughput, errors/429, CPU/RAM/connections	PENDING_DATA
Schema-per-tenant	Cùng tổ hợp và seed	Cùng metric per-tenant/per-run	PENDING_DATA
Silo database	Cùng tổ hợp và seed	Cùng metric per-tenant/per-run	PENDING_DATA

Bảng 4.5: Trạng thái đánh giá bổ sung

Nội dung	Artifact và điều kiện cần	Trạng thái
Noisy neighbor	Cặp limiter tắt/bật; aggressor/victim; metric tài nguyên	PENDING_DATA
Isolation/storage/payment	Hồ sơ loại, raw artifact hợp lệ, rubric/scorecard và review ADR	PENDING_DATA
Internet/provider	VPS/domain/TLS; callback/SMTP/VAPID thật theo scope	PENDING_DATA
Người dùng/SUS	Phạm vi xác nhận, đồng thuận, dữ liệu ẩn danh và codebook	PENDING_DATA

4.3.2 Giới hạn giá trị nội tại

Source inspection có thể bỏ sót đường gọi; tổng test không phản ánh toàn bộ permission matrix. Artifact cũ thiếu commit làm giảm provenance. Runtime state/credential và source cùng tên không bảo đảm cấu hình giống nhau. Database test bị skip khiến các bất biến SQL chưa được tái xác nhận hiện tại. Các cơ chế recovery/deduplication cần được kiểm tra ở đúng lớp, không suy từ claim provisioning sang claim outbox.

Đối với thực nghiệm, cache, JVM warm-up, I/O, thứ tự placement, seed/membership, pool và job nền đều có thể gây nhiễu. Công cụ thu metric cũng có thể thiếu tags hoặc sai cửa sổ. Phương án kiểm soát là khóa manifest, tách warm-up, xen kẽ thứ tự, lặp run, kiểm tra completeness và giữ raw input. Pilot chỉ phục vụ khóa protocol, không nhập vào kết quả cuối.

4.3.3 Giới hạn giá trị cấu trúc và ngoại tại

Test số lượng notification không đo việc người dùng nhận/đọc thông báo. p95 endpoint đọc không đại diện latency workflow Kanban hỗn hợp. VU không tự đồng nghĩa người dùng thật. HTTP 429 cần diễn giải theo policy thay vì gộp thành lỗi server. Những phân biệt này tránh dùng metric để thu làm đại diện cho chất lượng chưa được đo.

Nguyên mẫu Kanban trong đại học và một cấu hình VPS không đại diện mọi SaaS, database hoặc quy mô. Database riêng vẫn chung nhiều tài nguyên; cấu hình nhiều instance làm thay đổi limiter, quota lock và connection budget. Mailpit/FakePaymentProvider chỉ hỗ trợ luồng local. Availability/durability production, privacy certification và hiệu quả giáo dục cần đánh giá độc lập.

4.3.4 Nhất quán tài liệu và giới hạn phạm vi

Thời hạn hai khoảng tháng trong thuyết minh cần xác nhận bằng hồ sơ ký. Protocol 0.1 có note extension nhưng RQ/baseline đo vẫn Pool/Silo. ADR-0006 có thiết kế worker concurrency khác code tuần tự. Biên bản nhắc hạn có khác biệt tên file và ngày nội dung. Các mâu thuẫn được ghi trong baseline thay vì sửa hồi tổ protocol hoặc biên bản.

Capability, Custom Data, Approvals và Automation đã có code/local checkpoint, nhưng chưa phải phạm vi học thuật bổ sung đã xác nhận. Khi đưa vào phép đo, cần tách ảnh hưởng module khỏi placement; không để Silo chạy thêm automation rồi so độ trễ với Pool chỉ chạy lỗi. Bảng extension ở phụ lục mô tả khả năng hiện thực, không xác nhận tác động hoặc lựa chọn tối ưu.

4.3.5 Điều kiện hoàn tất bản cuối

Các khoảng trống đóng theo thứ tự: xác nhận thời hạn/phạm vi/quy cách; hoàn tất tổng quan có hệ thống nếu nhóm yêu cầu; chạy suite tích hợp hiện tại không skip; review hồ sơ loại/spike và ADR; khóa VPS/protocol, pilot và run độc lập; QA/sinh lại bảng; đánh giá người dùng khi được xác nhận. Mỗi kết luận mới phải có evidence ID và giới hạn tương ứng.

Bản tổng kết tối thiểu 50 trang nội dung theo tài liệu hình thức, không tính mục lục, bibliography và phụ lục. Số trang bản sơ bộ được báo cáo trong biên bản xuất bản; đạt hoặc chưa đạt độ dài không làm thay đổi trạng thái các cổng nghiên cứu. Công việc hoàn thiện nội dung sơ bộ không tự mở lại measurement, noisy-neighbor hoặc fault injection.

Kết luận và kiến nghị

Kết luận trong phạm vi bản sơ bộ

Báo cáo đã tổ chức bài toán đa thuê bao thành năm nhóm có truy vết từ tình huống, phương án, tiêu chí tới code, evidence và mục đánh giá. Nguyên mẫu Kanban hiện có account/control plane chung, verified tenant context, ba placement application data, quyền tenant/project, optimistic version, outbox và workflow provisioning. Đây là các mô tả hiện thực tại e829f73, được đối chiếu với source và hồ sơ local.

Lập luận sơ bộ cho thấy cần nhiều lớp bảo vệ và quản lý vòng đời: host/token không thay quyền project; namespace không thay runtime privilege; RLS không bảo vệ object storage; version không giải quyết delivery lặp; claim provisioning không chứng minh nhiều outbox worker an toàn. Các phân biệt này tạo tiêu chí kiểm chứng cụ thể thay cho tuyên bố an toàn hoặc tối ưu chung chung.

Bảng chứng được giữ theo checkpoint, môi trường và số skip. Sáu leak deliberate guard omission của hai ứng viên Pool được báo cáo như hành vi không an toàn; RLS + guard có căn cứ sàng lọc nhưng ADR selection vẫn Proposed. Kết quả unit/local không thay dữ liệu hiệu năng, pilot, SUS hoặc vận hành Internet. Bốn RQ có câu trả lời sơ bộ kèm giới hạn và điều kiện bổ sung trong Chương 4.

Chưa có cơ sở kết luận placement tối ưu, SLO đã đạt, efficacy limiter hoặc tác động giáo dục. Các bảng cần đo mang trạng thái PENDING_DATA. Báo cáo hoàn thiện nguồn và định dạng để phục vụ review, chưa là hồ sơ nghiệm thu hoặc kết luận nghiên cứu cuối.

Kiến nghị và công việc tiếp theo

Nhóm cần xác nhận thời hạn/phạm vi và phụ lục ba placement, rà tài liệu tham khảo, kiểm tra tích hợp không skip và review hồ sơ loại. Khi mở lại giai đoạn đo, cần khóa cấu hình VPS/workload/seed, pilot, chạy độc lập, lưu raw metric ngoài Git và sinh bảng/biểu đồ từ dữ liệu đã QA.

Các chủ đề kỹ thuật cần đánh giá riêng gồm nhiều API instance với limiter/quota, ngân sách connection toàn hệ thống, nhiều outbox worker, policy signed URL và fairness của worker tuần tự. Provider payment/SMTP/Web Push thật chỉ được kết luận sau contract và môi trường tương ứng. Nếu đánh giá người dùng thuộc phạm vi, chỉ thu sau xác nhận và đồng thuận; không gán điểm SUS hoặc tác động giả định.

Trước bản nộp cuối, cần kiểm tra số trang, bibliography, biểu mẫu và sản phẩm bàn giao. Chữ ký/nhận xét và thông tin chưa xác nhận dành cho người có thẩm quyền điền.

Tài liệu tham khảo

- [1] D. Olabanji, T. Fitch, and O. Matthew, “Multi-tenancy in cloud-native architecture: A systematic mapping study,” *WSEAS Transactions on Computers*, vol. 22, pp. 25–43, 2023. DOI: 10 . 37394 / 23205 . 2023 . 22 . 4. [Online]. Available: <https://wseas.com/journals/articles.php?id=7661> (visited on 10/09/2026).
- [2] V. R. Narasayya and S. Chaudhuri, “Cloud data services: Workloads, architectures and multi-tenancy,” *Foundations and Trends in Databases*, vol. 10, no. 1, pp. 1–107, 2021. DOI: 10 . 1561 / 19000000060. [Online]. Available: <https://www.nowpublishers.com/article/DownloadSummary/DBS-060> (visited on 10/09/2026).
- [3] Amazon Web Services. “SaaS lens: Targeted isolation,” [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/targeted-isolation.html> (visited on 10/09/2026).
- [4] Microsoft. “Map requests to tenants in a multitenant solution,” [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/map-requests> (visited on 10/09/2026).
- [5] Amazon Web Services. “Transactional outbox pattern,” [Online]. Available: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html> (visited on 10/09/2026).
- [6] K. Petersen, S. Vakkalanka, and L. Kuzniarz, “Guidelines for conducting systematic mapping studies in software engineering: An update,” *Information and Software Technology*, vol. 64, pp. 1–18, 2015. DOI: 10.1016/j.infsof.2015.03.007. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950584915000646> (visited on 10/09/2026).
- [7] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, *et al.*, “The PRISMA 2020 statement: An updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, n71, 2021. DOI: 10 . 1136 / bmj . n71. [Online]. Available: <https://www.bmj.com/content/372/bmj.n71.long> (visited on 10/09/2026).

- [8] S. Pushpan, “Multi-tenant architecture: A comprehensive framework for building scalable SaaS applications,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 10, no. 6, pp. 1117–1126, 2024, Metadata verified; quality appraisal pending. DOI: 10 . 32628 / CSEIT241061151. [Online]. Available: <https://ijsrcseit.com/index.php/home/article/view/CSEIT241061151> (visited on 10/09/2026).
- [9] Amazon Web Services. “SaaS lens: Noisy neighbor,” [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/noisy-neighbor.html> (visited on 10/09/2026).
- [10] PostgreSQL Global Development Group. “PostgreSQL 18 documentation: Row security policies,” [Online]. Available: <https://www.postgresql.org/docs/18/ddl-rowsecurity.html> (visited on 10/09/2026).
- [11] PostgreSQL Global Development Group. “PostgreSQL 18 documentation: Schemas,” [Online]. Available: <https://www.postgresql.org/docs/18/ddl-schemas.html> (visited on 10/09/2026).
- [12] Microsoft. “Architectural considerations for identity in a multitenant solution,” [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/identity> (visited on 10/09/2026).

Phụ lục A

Tái lập và kiểm toán bằng chứng

A.1 Baseline, nguồn và sổ bằng chứng

Các đường dẫn dưới đây tương đối với root repository. Commit nền là e829f73; thay đổi report chưa commit được nhận diện riêng. Sổ đầy đủ ở docs/research/preliminary-report-evidence.md, ma trận sáu bước ở docs/research/preliminary-report-traceability.md và biên bản kiểm tra ở docs/testing/preliminary-report-2026-10-09.md. Raw measurements và dữ liệu người tham gia không được đưa vào Git trong công việc này.

Bảng A.1: Sổ bằng chứng và giới hạn kiểm toán

Mã	Mệnh đề, nguồn và provenance
E01	Baseline và nguồn yêu cầu. SOURCE. Ngày: 2026-10-09. Phiên bản: e829f73. Nguồn: resource/thuyetMinhSaasMultiTenancy.md; docs/research/srs.md; danh sách đầy đủ trong sổ Markdown. Kết quả: Checkout main; working tree report đã có thay đổi. Thuyết minh ghi hai khoảng thời gian khác nhau. Skip: Không áp dụng. Giới hạn: Yêu cầu/phạm vi từ hồ sơ; thời hạn và phần mở rộng học thuật chưa được xác nhận. Không coi status runtime cũ là hiện tại.
E02	Protocol và quản trị nghiên cứu. SOURCE. Ngày: 2026-08-25; đọc lại 2026-10-09. Phiên bản: UNESTABLISHED. Nguồn: docs/research/protocol.md; docs/research/literature-search-log.md; danh sách đầy đủ trong sổ Markdown. Kết quả: Protocol 0.1 giữ RQ1–RQ4 và baseline Pool/Silo; note yêu cầu phụ lục cho phép đo ba placement. Skip: Không áp dụng. Giới hạn: Chưa khóa tìm kiếm và chưa có dữ liệu người dùng; báo cáo không sửa hồi tố protocol.

Mã	Mệnh đề, nguồn và provenance
E03	Artifact Surefire có sẵn trước lượt viết. ARTIFACT. Ngày: Quan sát 2026-10-09; mtime 2026-09-24 Asia/Taipei. Phiên bản: UNESTABLISHED. Nguồn: apps/api/target/surefire-reports. Kết quả: Artifact cũ 106 test, 0 failure/error, 47 skip: 59 thực thi. Đã lưu bản sao trong /tmp trước kiểm tra mới. Skip: 47: Docker không khả dụng. Giới hạn: Mtime không chứng minh ngày chạy hoặc commit; XML trong target sau lượt mới đã bị thay bằng kết quả E04.
E04	Kiểm tra backend mới và môi trường hiện tại. NEW_TECHNICAL_CHECK. Ngày: 2026-10-09 11:13:19 +08:00. Phiên bản: e829f73; không có diff apps. Nguồn: docs/testing/preliminary-report-2026-10-09.md; docs/testing/preliminary-report-2026-10-09.json. Kết quả: Fresh compile 121 source + 27 test source; test goal đạt: 106 tổng, 59 thực thi, 47 skip, 0 failure/error. Maven/test runtime Java 21.0.12.1; compiler javac 25.0.4.1 fork với -release 21. Skip: 47: disabledWithoutDocker. Giới hạn: Verify chưa đạt do thiếu Maven plugin offline. Compiler trong runtime 21 báo release 21 unsupported, nên dùng compiler fork đã ghi rõ. Frontend/E2E không chạy vì thiếu Node/runtime stack.
E05	Checkpoint EXT ba placement. HISTORICAL. Ngày: 2026-09-08 UTC. Phiên bản: Working tree sau c226676; không gán HEAD hiện tại. Nguồn: docs/testing/extension-local-2026-09-08.md. Kết quả: Application V8, backend 96/96 (skip 0), frontend 16, Playwright 3 theo biên bản; schema isolation/upgrade local. Skip: Backend 0 theo biên bản. Giới hạn: Kết quả lịch sử local; extension không tự là phạm vi học thuật đã duyệt hoặc runtime hôm nay.

Mã	Mệnh đề, nguồn và provenance
E06	Checkpoint nhắc deadline V12. HISTORICAL. Ngày: Nội dung 2026-09-19 UTC+7; tên file 2026-09-18. Phiên bản: an_upgrade_features theo biên bản; SHA chưa ghi. Nguồn: docs/testing/deadline-reminders-2026-09-18.md. Kết quả: Biên bản ghi deadline integration 6/6, backend 106, frontend 22, E2E 3, application V12; kiểm tra stale và exact assignee. Skip: Tổng skip không ghi riêng. Giới hạn: Phân biệt ngày nội dung/tên file; không suy 106/106 pass hiện tại, không có SMTP Internet/VAPID thật.
E07	P0 baseline local. HISTORICAL. Ngày: 2026-08-26. Phiên bản: Theo biên bản; không liên kết HEAD hiện tại. Nguồn: docs/testing/p0-verification-2026-08-26.md. Kết quả: Baseline Pool/Silo, migration/role/RLS, API/Compose và k6 smoke theo hồ sơ. Skip: Theo từng lệnh trong biên bản. Giới hạn: Smoke timing không phải số đo nghiên cứu; trạng thái stack khi chốt không được giả định còn nguyên.
E08	P1 provisioning và storage recovery. HISTORICAL. Ngày: 2026-08-31. Phiên bản: Theo từng biên bản, không gán một SHA chung. Nguồn: docs/testing/p1-verification-2026-08-31.md; docs/testing/p1-verification-2026-08-31-part-2.md. Kết quả: Hồ sơ ghi MinIO dead letter attempt 5/requeue riêng Pool-Silo, force-kill JVM sau các ranh giới, lease recovery, rollback lỗi lặp và manual retry. Skip: Theo từng lệnh trong biên bản. Giới hạn: Không chạy lại fault injection trong lượt viết; không đại diện availability/durability production.

Mã	Mệnh đề, nguồn và provenance
E09	Sàng lọc guard-omission isolation. HISTORICAL_SCREENING. Ngày: 2026-08-31. Phiên bản: Nền harness 2b430b7 theo ADR; quan sát lưu trong biên bản lượt 3. Nguồn: docs/testing/p2-preparation-2026-08-31-part-3.md; docs/architecture/adrs/0003-pool-isolation-selection.md; danh sách đầy đủ trong sổ Markdown. Kết quả: Guarded contract 6/6; 18 observation: 6 explicit/Hibernate Pool leak, 3 RLS Pool protected, 9 Silo boundary protected. Skip: Theo hồ sơ; không chạy harness mới. Giới hạn: Harness pass là phân loại đúng, không đổi leak thành PASS. Measurement/scorecard/hồ sơ loại checksum-backed chưa hoàn tất.
E10	N1 tenant context. SOURCE_AND_TEST_DESIGN. Ngày: 2026-10-09. Phiên bản: e829f73. Nguồn: apps/api/src/main/java/vn/edu/ctu/saas/security/TenantContextFilter.java; apps/api/src/main/java/vn/edu/ctu/saas/security/TenantHostResolver.java; danh sách đầy đủ trong sổ Markdown. Kết quả: Host/token/status/membership version binding; ThreadLocal finally cleanup; transaction-local set_config và search_path. Unit test hiện có chạy trong E04. Skip: Xem E04. Giới hạn: Source mô tả cơ chế; chưa có trusted proxy/DNS/TLS Internet được nghiệm thu.

Mã	Mệnh đề, nguồn và provenance
E11	N2 datasource và isolation. SOURCE_AND_TEST_DESIGN. Ngày: 2026-10-09. Phiên bản: e829f73. Nguồn: apps/api/src/main/java/vn/edu/ctu/saas/tenant/DefaultTenantDataSourceResolver.java; apps/api/src/main/java/vn/edu/ctu/saas/provisioning/TenantDatabaseProvisioner.java; danh sách đầy đủ trong sổ Markdown. Kết quả: Ba placement; runtime role/schema/database, migration V1–V12 và guard/RLS. Skip: RLS 4 + schema 1 skip trong E04. Giới hạn: Runtime privilege cần kiểm tra trên database thật; source không chứng minh mọi deployment đúng.
E12	N3 identity và authorization. SOURCE_AND_TEST_DESIGN. Ngày: 2026-10-09. Phiên bản: e829f73. Nguồn: apps/api/src/main/java/vn/edu/ctu/saas/auth/AuthService.java; apps/api/src/main/java/vn/edu/ctu/saas/application/ProjectApplicationService.java; danh sách đầy đủ trong sổ Markdown. Kết quả: Account chung, tenant-bound session, active membership/security version và project role; deny/IDOR assertions hiện có. Skip: 17 project integration skip; management unit thực thi. Giới hạn: Chỉ kết luận các case có evidence; không chứng nhận identity production hoặc mọi ô policy.

Mã	Mệnh đề, nguồn và provenance
E13	N4 version, outbox, reminder và delivery. SOURCE_AND_TEST_DESIGN. Ngày: 2026-10-09. Phiên bản: e829f73. Nguồn: apps/api/src/main/java/vn/edu/ctu/saas/application/ProjectApplicationService.java; apps/api/src/main/java/vn/edu/ctu/saas/notification/OutboxWorker.java; danh sách đầy đủ trong sổ Markdown. Kết quả: Version predicate, batch transaction/outbox, sổ nhắc hạn, stale recheck và email queue riêng; outbox duyệt tenant tuần tự. Skip: 6 deadline integration skip trong E04. Giới hạn: Không exactly-once SMTP; không kế thừa SKIP LOCKED provisioning cho outbox; multi-worker còn thiếu.
E14	N5 provisioning và tài nguyên. SOURCE_AND_TEST_DESIGN. Ngày: 2026-10-09. Phiên bản: e829f73. Nguồn: apps/api/src/main/java/vn/edu/ctu/saas/provisioning/ProvisioningJobClaimer.java; apps/api/src/main/java/vn/edu/ctu/saas/provisioning/ProvisioningJobCoordinator.java; danh sách đầy đủ trong sổ Markdown. Kết quả: Atomic claim/lease, compensation, datasource eviction, process-local quota lock và tenant bucket limiter. Skip: Integration Docker xem E04. Giới hạn: Chưa đo global connection budget, quota/limiter multi-instance hoặc provider thật; pool cap logic chưa là proof hard bound toàn hệ thống.

Mã	Mệnh đề, nguồn và provenance
E15	Contract, architecture và workload. SOURCE. Ngày: 2026-10-09. Phiên bản: e829f73. Nguồn: docs/api/openapi.yaml; docs/architecture/overview.md; danh sách đầy đủ trong sổ Markdown. Kết quả: API chung; baseline constant-vus + think time; load constant-arrival-rate. ADR thiết kế concurrency khác worker tuần tự. Skip: Không áp dụng. Giới hạn: Workload đọc chưa là mixed Kanban; protocol ba placement và capability cần đăng ký bổ sung.
E16	Frontend và E2E lịch sử. HISTORICAL_AND_ENVIRONMENT. Ngày: 08/09 và 19/09; môi trường đọc lại 09/10/2026. Phiên bản: Theo E05/E06. Nguồn: apps/web/package.json; docs/testing/extension-local-2026-09-08.md; danh sách đầy đủ trong sổ Markdown. Kết quả: 16 frontend test ở EXT, 22 ở mốc sau; 3 E2E theo hồ sơ. Node Linux không có trong môi trường hiện tại. Skip: Không có lượt frontend/E2E mới. Giới hạn: Không tuyên bố contract/lint/test/build hoặc E2E đã chạy lại hôm nay.
E17	Xuất bản LaTeX/PDF sơ bộ. NEW_PUBLISHING_CHECK. Ngày: 2026-10-09. Phiên bản: e829f73 + working tree report. Nguồn: report/PRELIMINARY_VALIDATION.md; report/Makefile; danh sách đầy đủ trong sổ Markdown. Kết quả: Doctor, all smoke và check đạt; review trực quan được ghi trong biên bản xuất bản. Skip: Không áp dụng. Giới hạn: Định dạng/build không thay công nghiên cứu, đủ trang nội dung hoặc nghiệm thu. Kết quả cuối đọc tại biên bản xuất bản.

A.2 Môi trường và lệnh tái lập xuất bản

Yêu cầu công cụ gồm Python, Make, XeLaTeX, Biber/BibTeX, Poppler và Times New Roman đủ bốn kiểu. Lệnh thực hiện từ root:

```
make -C report doctor
make -C report all smoke
make -C report check
```

Đầu ra chính ở `report/build/content/content.pdf` và `report/build/content-twoside/content.pdf`. Bốn bản tin/tóm tắt và smoke có thư mục build riêng. BibTeX trong examples chỉ là tài nguyên kiểm tra định dạng; nguồn khoa học của báo cáo nằm ở bibliography và được trích dẫn trong nội dung chính.

A.3 Tái lập kiểm tra kỹ thuật và điều kiện

Backend verification chuẩn dùng Java 21 và Maven wrapper với mục tiêu verify. Frontend cần Node ít nhất 22.12, dependency đúng lockfile, rồi contract/lint/test/build. Testcontainers cần Docker; E2E cần stack và environment đúng. Không giả định môi trường hiện tại có các điều kiện ấy chỉ vì một checkpoint trước đây đã pass.

Lượt viết này dùng cache Maven sao chép trong thư mục tạm để tránh ghi home read-only. Các lệnh và kết quả thực tế, gồm mục tiêu nào thất bại và test nào skip, được lưu ở biên bản E04. Để đóng baseline cuối cần chạy lại verify và frontend đầy đủ trên môi trường chuẩn, kèm commit, ngày, số test và skipped count.

A.4 Manifest cho đợt thực nghiệm sau

Một manifest tối thiểu ghi run ID/class, protocol ID, commit/image digest, working-tree state, seed, tenant/account/project/task counts, placement, VU hoặc arrival rate, think time, warm-up/cửa sổ đo, pool/limiter và phiên bản công cụ. Phần cứng, dịch vụ chạy chung, metric nguồn và checksum phải đi cùng raw artifact. Thông tin token/secret không nằm trong manifest.

Kết quả derived được sinh từ raw input đã QA. Một run development/pilot không đổi nhãn thành experiment; mọi điều chỉnh protocol phải ghi trước. Các bảng pending trong Chương 4 không được điền tay từ ảnh dashboard hoặc từ một smoke iteration.

A.5 Checklist xuất bản và nội dung

Checklist gồm đủ Mở đầu/bốn chương/Kết luận/bibliography/phụ lục; N1–N5 có đủ chuỗi sáu bước; mỗi RQ có bằng chứng/giới hạn/công việc bổ sung; citation/reference ổn định; bảng dài lặp header; năm sơ đồ đọc được; Times New Roman nhúng và A4. Kiểm tra tự động không thay xem trực quan bìa, mục lục, hình và bibliography. Số trang chính và mức đáp ứng bản cuối được ghi riêng trong biên bản xuất bản.

Phụ lục B

Ma trận quyền, mở rộng và công việc còn thiếu

B.1 Ma trận quyền rút gọn

Ma trận này tóm tắt một số hành động để đọc báo cáo; nguồn policy đầy đủ là docs/research/permission-matrix.md. A: cho phép khi các điều kiện tenant/project hiện hành hợp lệ; D: role đang xét không tự cấp quyền; C: có điều kiện ownership hoặc quan hệ cụ thể. Tenant role và project role không được gộp thành một thang quyền toàn hệ thống.

Bảng B.1: Phân biệt quyền tenant và quyền project

Hành động cấp tenant	Owner	Admin	Member
Mời/quản lý member trong giới hạn policy	A	C	D
Chuyển ownership	A	D	D
Xem aggregate storage usage	A	A	D
Tạo project	A	A	A
Đọc nội dung project nhờ tenant role	D	D	D

Bảng B.2: Một số quyền nội dung project

Hành động	Manager	Member	Viewer
Đọc project/board/task/comment	A	A	A
Quản lý board/column/project member	A	D	D
Tạo/sửa/move task, bình luận	A	A	D
Xóa task	A	D	D
Tải resource đã authorize	A	A	A
Xóa resource theo ownership/policy	A	C	D

Mỗi hành động cần kiểm tra role thấp nhất được allow, role bị deny, membership thiếu/revoked, ID tenant khác, host/token mismatch, suspended tenant và trạng thái dữ liệu không đổi sau deny. Archived project còn có read-only invariant. Bảng quyền mô tả policy; số ô đã kiểm chứng phải truy tới assertion/lần chạy, không suy từ A/D trong bảng.

B.2 Phần mở rộng ba placement và capability

Code/local checkpoint có capability grant/enable theo server policy, độc lập với tier. Bảng dưới mô tả khả năng cấp, không mô tả kết quả nghiên cứu đã phê duyệt. Effective capability còn phụ thuộc grant, enable và role hiện hành. Custom Data tạo field/bảng

Bảng B.3: Capability có thể cấp theo placement local

Placement	Branding	Custom Data	Approvals	Automation
Pool	Có	Không	Không	Không
Schema-per-tenant	Có	Có	Không	Không
Silo database	Có	Có	Có	Có

hữu hạn qua schema job; Approvals lưu snapshot và version; Automation có tập trigger/action hữu hạn và idempotency. Những module này không được dùng làm biến nhiễu trong comparison nghiệp vụ lỗi. Thu hồi capability phải tuân policy giữ dữ liệu/ lịch sử; capability không bỏ qua tenant/project authorization.

B.3 Các mâu thuẫn đã nhận diện

Các điểm cần xác nhận gồm thời gian 05–10 so với 03–08/2026; nhánh/nền checkpoint trong PROJECT_STATUS so với main hiện tại; V8/96/16 so với V12/106/22 ở mốc sau; ngày nội dung biên bản nhắc hạn khác tên file; protocol baseline Pool/Silo so với extension; worker concurrency trong ADR khác code tuần tự; IEEE so với quy cách cuối. Tham chiếu resource/plan.md đã mất được thay trong báo cáo bằng hồ sơ hiện hữu, không khôi phục file.

B.4 Điều kiện đóng các khoảng trống

Bảng B.4: Điều kiện bổ sung trước kết luận cuối

Khoảng trống	Điều kiện đóng
Thời hạn và phạm vi	Hồ sơ ký/xác nhận học thuật, gồm extension và comparison ba placement
Tổng quan	Ngày tìm kiếm, export, dedup/screening/extraction và chất lượng nguồn có truy vết

Khoảng trống	Điều kiện đóng
Tích hợp hiện tại	Verify/front-end/E2E đúng phiên bản; Docker suite thực thi; skipped count rõ
Lựa chọn công nghệ	Hồ sơ loại và measured artifact/scorecard review; ADR rõ trạng thái
Hiệu năng/noisy neighbor	Protocol khóa, VPS/pilot/run lặp, per-tenant metric và QA raw data
Nhiều process/worker	Test limiter/quota/cache budget/dedupe trên đúng topology multi-instance
Internet/provider	Domain/TLS, payment/SMTP/VAPID theo scope cùng artifact kiểm chứng
Người dùng	Phạm vi được xác nhận, tuyển mẫu/đồng thuận và dữ liệu ẩn danh
Hồ sơ nộp	Số trang nội dung, bibliography, biểu mẫu và sản phẩm bàn giao được xác nhận

Không đánh dấu cổng A/B/E hoặc nghiệm thu cuối từ việc hoàn thiện bản sơ bộ. Các khoảng trống trên không cản việc review nội dung kiến trúc, nhưng giới hạn loại kết luận có thể đưa ra.